

CHƯƠNG 12. MÔ HÌNH TÙY CHỈNH VÀ HUẤN LUYỆN VỚI TENSORFLOW

Cho đến nay, chúng ta chỉ sử dụng API cấp cao của TensorFlow, Keras, nhưng nó đã giúp chúng ta đi được khá xa: chúng ta đã xây dựng nhiều kiến trúc mạng nơ-ron khác nhau, bao gồm mạng hồi quy và phân loại, mạng Rộng & Sâu, và mạng tự chuẩn hóa, sử dụng đủ loại kỹ thuật, chẳng hạn như chuẩn hóa theo batch, dropout và lịch trình tốc độ học. Thực tế, 95% các trường hợp sử dụng bạn sẽ gặp phải sẽ không yêu cầu bất cứ điều gì ngoài Keras (và `tf.data`; xem Chương 13). Nhưng bây giờ là lúc để đi sâu hơn vào TensorFlow và xem xét API Python cấp thấp hơn của nó. Điều này sẽ hữu ích khi bạn cần kiểm soát thêm để viết các hàm mất mát tùy chỉnh, số liệu tùy chỉnh, lớp, mô hình, trình khởi tạo, trình chính quy hóa, ràng buộc trọng số và hơn thế nữa. Bạn thậm chí có thể cần kiểm soát hoàn toàn vòng lặp huấn luyện; ví dụ, để áp dụng các phép biến đổi hoặc ràng buộc đặc biệt cho các gradient (ngoài việc chỉ cắt chúng) hoặc để sử dụng nhiều bộ tối ưu hóa cho các phần khác nhau của mạng. Chúng ta sẽ đề cập đến tất cả các trường hợp này trong chương này, và chúng ta cũng sẽ xem xét cách bạn có thể tăng cường các mô hình tùy chỉnh và thuật toán huấn luyện của mình bằng tính năng tạo biểu đồ tự động của TensorFlow. Nhưng trước tiên, hãy cùng khám phá nhanh TensorFlow.

Giới thiệu nhanh về TensorFlow

Như bạn đã biết, TensorFlow là một thư viện mạnh mẽ cho tính toán số, đặc biệt phù hợp và được tinh chỉnh cho học máy quy mô lớn (nhưng bạn có thể sử dụng nó cho bất cứ thứ gì khác yêu cầu tính toán nặng). Nó được phát triển bởi nhóm Google Brain và nó cung cấp năng lượng cho nhiều dịch vụ quy mô lớn của Google, chẳng hạn như Google Cloud Speech, Google Photos và Google Search. Nó được mã nguồn mở vào tháng 11 năm 2015, và hiện là thư viện học sâu được sử dụng rộng rãi nhất trong ngành. Vô số dự án sử dụng TensorFlow cho tất cả các loại tác vụ học máy, chẳng hạn như phân loại hình ảnh, xử lý ngôn ngữ tự nhiên, hệ thống đề xuất và dự báo chuỗi thời gian.

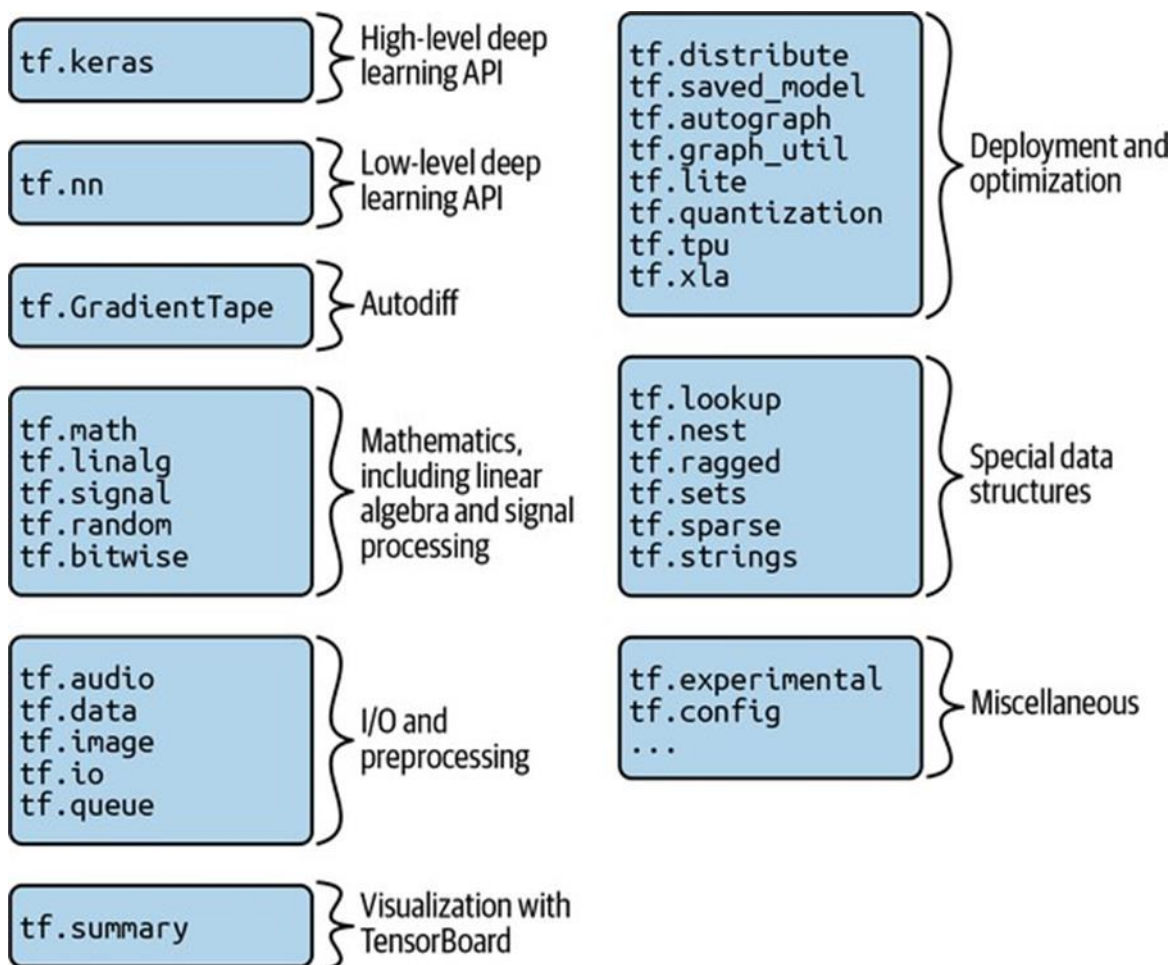
Vậy TensorFlow cung cấp những gì? Dưới đây là tóm tắt:

- Cốt lõi của nó rất giống NumPy, nhưng có hỗ trợ GPU.
- Nó hỗ trợ tính toán phân tán (trên nhiều thiết bị và máy chủ).
- Nó bao gồm một loại trình biên dịch just-in-time (JIT) cho phép nó tối ưu hóa các phép tính về tốc độ và sử dụng bộ nhớ. Nó hoạt động bằng cách trích xuất biểu đồ tính toán từ một hàm Python, tối ưu hóa nó (ví dụ: bằng cách cắt các nút không sử dụng), và chạy nó một cách hiệu quả (ví dụ: bằng cách tự động chạy các hoạt động độc lập song song).
- Biểu đồ tính toán có thể được xuất sang một định dạng di động, vì vậy bạn có thể huấn luyện một mô hình TensorFlow trong một môi trường (ví dụ: sử dụng Python trên Linux) và chạy nó trong một môi trường khác (ví dụ: sử dụng Java trên thiết bị Android).

- Nó triển khai tự động đạo hàm ngược (xem Chương 10 và Phụ lục B) và cung cấp một số bộ tối ưu hóa xuất sắc, chẳng hạn như RMSProp và Nadam (xem Chương 11), vì vậy bạn có thể dễ dàng giảm thiểu tất cả các loại hàm mất mát.

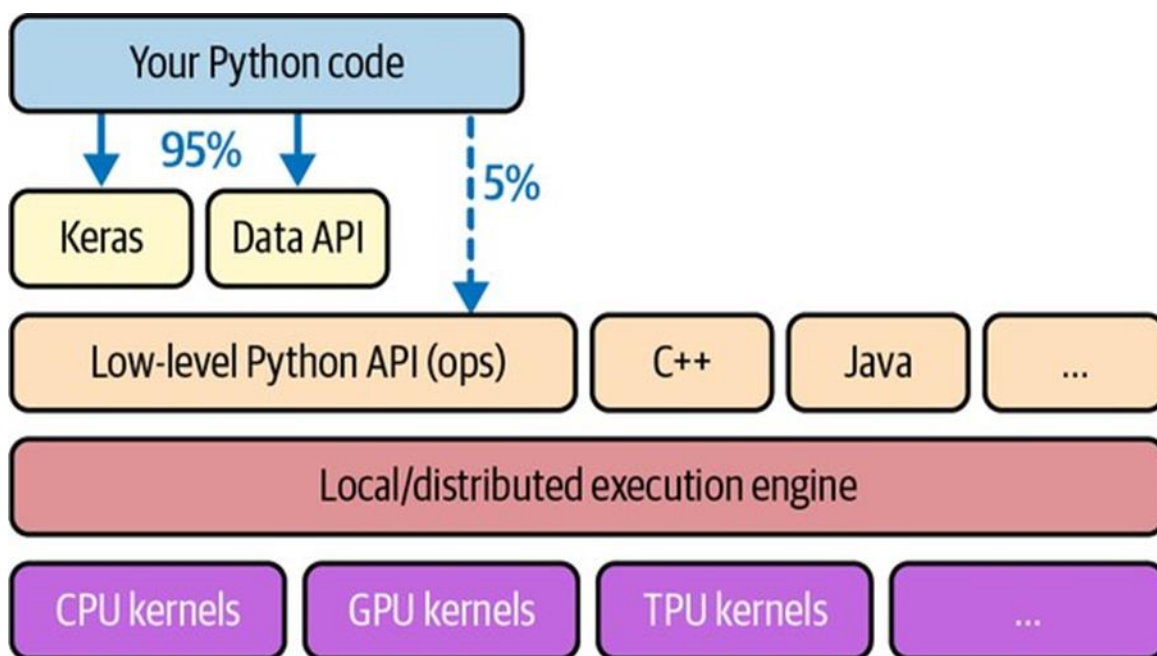
TensorFlow cung cấp nhiều tính năng hơn được xây dựng trên các tính năng cốt lõi này: quan trọng nhất tất nhiên là Keras, nhưng nó cũng có các phép toán tải và tiền xử lý dữ liệu (`tf.data`, `tf.io`, v.v.), các phép toán xử lý hình ảnh (`tf.image`), các phép toán xử lý tín hiệu (`tf.signal`), và hơn thế nữa (xem Hình 12-1 để biết tổng quan về API Python của TensorFlow).

Ở cấp độ thấp nhất, mỗi phép toán TensorFlow (op) được triển khai bằng mã C++ hiệu quả cao. Nhiều phép toán có nhiều triển khai được gọi là `kernel`s: mỗi `kernel` dành riêng cho một loại thiết bị cụ thể, chẳng hạn như CPU, GPU hoặc thậm chí TPU (đơn vị xử lý tensor). Như bạn có thể biết, GPU có thể tăng tốc đáng kể các phép tính bằng cách chia chúng thành nhiều phần nhỏ hơn và chạy chúng song song trên nhiều luồng GPU. TPU thậm chí còn nhanh hơn: chúng là chip ASIC tùy chỉnh được xây dựng đặc biệt cho các phép toán học sâu (chúng ta sẽ thảo luận cách sử dụng TensorFlow với GPU hoặc TPU trong Chương 19).



Hình 12-1. API Python của TensorFlow

Kiến trúc của TensorFlow được thể hiện trong Hình 12-2. Hầu hết thời gian mã của bạn sẽ sử dụng các API cấp cao (đặc biệt là Keras và `tf.data`), nhưng khi bạn cần linh hoạt hơn, bạn sẽ sử dụng API Python cấp thấp hơn, xử lý trực tiếp các tensor. Trong mọi trường hợp, công cụ thực thi của TensorFlow sẽ đảm nhiệm việc chạy các hoạt động một cách hiệu quả, ngay cả trên nhiều thiết bị và máy nếu bạn yêu cầu. TensorFlow chạy không chỉ trên Windows, Linux và macOS, mà còn trên thiết bị di động (sử dụng TensorFlow Lite), bao gồm cả iOS và Android (xem Chương 19). Lưu ý rằng API cho các ngôn ngữ khác cũng có sẵn, nếu bạn không muốn sử dụng API Python: có API C++, Java và Swift. Thậm chí còn có một triển khai JavaScript được gọi là TensorFlow.js cho phép chạy các mô hình của bạn trực tiếp trong trình duyệt.



Hình 12-2. Kiến trúc của TensorFlow

Có nhiều hơn nữa về TensorFlow ngoài thư viện. TensorFlow là trung tâm của một hệ sinh thái thư viện rộng lớn. Đầu tiên, có TensorBoard để trực quan hóa (xem Chương 10). Tiếp theo, có TensorFlow Extended (TFX), là một tập hợp các thư viện được Google xây dựng để đưa các dự án TensorFlow vào sản xuất: nó bao gồm các công cụ để xác thực dữ liệu, tiền xử lý, phân tích mô hình và phục vụ (với TF Serving; xem Chương 19). TensorFlow Hub của Google cung cấp một cách để dễ dàng tải xuống và tái sử dụng các mạng nơ-ron đã được huấn luyện trước. Bạn cũng có thể nhận được nhiều kiến trúc mạng nơ-ron, một số trong đó đã được huấn luyện trước, trong kho mô hình của TensorFlow. Kiểm tra Tài nguyên TensorFlow và <https://github.com/jtoy/awesome-tensorflow> để biết thêm các dự án dựa trên TensorFlow. Bạn sẽ tìm thấy hàng trăm dự án TensorFlow trên GitHub, vì vậy việc tìm mã hiện có cho bất cứ điều gì bạn đang cố gắng làm thường rất dễ dàng.

Cuối cùng nhưng không kém phần quan trọng, TensorFlow có một nhóm các nhà phát triển nhiệt tình và hữu ích tận tâm, cũng như một cộng đồng lớn đóng góp vào việc cải thiện nó. Để hỏi các câu hỏi kỹ thuật, bạn nên sử dụng <https://stackoverflow.com> và gắn thẻ câu hỏi

của bạn với tensorflow và python. Bạn có thể gửi lỗi và yêu cầu tính năng thông qua GitHub. Đối với các cuộc thảo luận chung, hãy tham gia Diễn đàn TensorFlow. OK, đã đến lúc bắt đầu viết mã!

Sử dụng TensorFlow như NumPy

API của TensorFlow xoay quanh các tensor, chúng chảy từ phép toán này sang phép toán khác – do đó có tên TensorFlow. Một tensor rất giống với ndarray của NumPy: nó thường là một mảng đa chiều, nhưng nó cũng có thể chứa một đại lượng vô hướng (một giá trị đơn giản, chẳng hạn như 42). Các tensor này sẽ rất quan trọng khi chúng ta tạo các hàm chi phí tùy chỉnh, các số liệu tùy chỉnh, các lớp tùy chỉnh và hơn thế nữa, vì vậy hãy xem cách tạo và thao tác với chúng.

Tensor và các phép toán

Bạn có thể tạo một tensor với `tf.constant()`. Ví dụ, đây là một tensor đại diện cho một ma trận với hai hàng và ba cột số thập phân:

```
>>> import tensorflow as tf

>>> t = tf.constant([[1., 2., 3.], [4., 5., 6.]]) # matrix

>>> t
<tf.Tensor: shape=(2, 3), dtype=float32, numpy=
array([[1., 2., 3.],
       [4., 5., 6.]], dtype=float32)>
```

Giống như một ndarray, một `tf.Tensor` có hình dạng (shape) và kiểu dữ liệu (dtype):

```
>>> t.shape
TensorShape([2, 3])

>>> t.dtype
tf.float32
```

Đánh chỉ mục hoạt động rất giống trong NumPy:

```
>>> t[:, 1:]
<tf.Tensor: shape=(2, 2), dtype=float32, numpy=
array([[2., 3.],
       [5., 6.]], dtype=float32)>

>>> t[..., 1, tf.newaxis]
<tf.Tensor: shape=(2, 1), dtype=float32, numpy=
array([[2.],
       [5.]], dtype=float32)>
```

Quan trọng nhất, tất cả các loại phép toán tensor đều có sẵn:

```
>>> t + 10
<tf.Tensor: shape=(2, 3), dtype=float32, numpy=
array([[11., 12., 13.],
       [14., 15., 16.]], dtype=float32)>

>>> tf.square(t)
<tf.Tensor: shape=(2, 3), dtype=float32, numpy=
array([[ 1.,  4.,  9.],
       [16., 25., 36.]], dtype=float32)>

>>> t @ tf.transpose(t)
<tf.Tensor: shape=(2, 2), dtype=float32, numpy=
array([[14., 32.],
       [32., 77.]], dtype=float32)>
```

Lưu ý rằng viết `t + 10` tương đương với gọi `tf.add(t, 10)` (thực tế, Python gọi phương thức magic `t.__add__(10)`, cái này chỉ gọi `tf.add(t, 10)`). Các toán tử khác, như `-` và `*`, cũng được hỗ trợ. Toán tử `@` đã được thêm vào Python 3.5, cho phép nhân ma trận: nó tương đương với việc gọi hàm `tf.matmul()`.

Một tensor cũng có thể chứa một giá trị vô hướng. Trong trường hợp này, hình dạng là trống:

```
>>> tf.constant(42)
<tf.Tensor: shape=(), dtype=int32, numpy=42>
```

Bạn sẽ tìm thấy tất cả các phép toán cơ bản bạn cần (`tf.add()`, `tf.multiply()`, `tf.square()`, `tf.exp()`, `tf.sqrt()`, v.v.) và hầu hết các phép toán mà bạn có thể tìm thấy trong NumPy (ví dụ: `tf.reshape()`, `tf.squeeze()`, `tf.tile()`). Một số hàm có tên khác với trong NumPy; ví dụ, `tf.reduce_mean()`, `tf.reduce_sum()`, `tf.reduce_max()`, và `tf.math.log()` tương đương với `np.mean()`, `np.sum()`, `np.max()`, và `np.log()`. Khi tên khác nhau, thường có một lý do chính đáng. Ví dụ, trong TensorFlow bạn phải viết `tf.transpose(t)`; bạn không thể chỉ viết `t.T` như trong NumPy. Lý do là hàm `tf.transpose()` không làm chính xác những gì thuộc tính `T` của NumPy làm: trong TensorFlow, một tensor mới được tạo với bản sao riêng của dữ liệu được chuyển vị, trong khi trong NumPy, `t.T` chỉ là một chế độ xem chuyển vị trên cùng dữ liệu. Tương tự, phép toán `tf.reduce_sum()` được đặt tên như vậy vì kernel GPU của nó (tức là triển khai GPU) sử dụng thuật toán giảm không đảm bảo thứ tự các phần tử được thêm vào: vì số thập phân 32 bit có độ chính xác hạn chế, kết quả có thể thay đổi rất nhỏ mỗi khi bạn gọi phép toán này. Điều tương tự cũng đúng với `tf.reduce_mean()` (nhưng tất nhiên `tf.reduce_max()` là xác định).

Tensor và NumPy

Các tensor tương thích tốt với NumPy: bạn có thể tạo một tensor từ một mảng NumPy và ngược lại. Bạn thậm chí có thể áp dụng các phép toán TensorFlow cho mảng NumPy và các phép toán NumPy cho tensor:

```
>>> import numpy as np
```

```
>>> a = np.array([2., 4., 5.])

>>> tf.constant(a)
<tf.Tensor: id=111, shape=(3,), dtype=float64, numpy=array([2., 4., 5.])>

>>> t.numpy() # hoặc np.array(t)
array([[1., 2., 3.],
       [4., 5., 6.]], dtype=float32)

>>> tf.square(a)

<tf.Tensor: id=116, shape=(3,), dtype=float64, numpy=array([4., 16., 25.])>

>>> np.square(t)
array([[ 1.,  4.,  9.],
       [16., 25., 36.]], dtype=float32)
```

Chuyển đổi kiểu dữ liệu

Chuyển đổi kiểu dữ liệu có thể làm giảm hiệu suất đáng kể và chúng có thể dễ dàng bị bỏ qua khi được thực hiện tự động. Để tránh điều này, TensorFlow không tự động thực hiện bất kỳ chuyển đổi kiểu nào: nó chỉ đưa ra một ngoại lệ nếu bạn cố gắng thực hiện một phép toán trên các tensor có kiểu không tương thích. Ví dụ, bạn không thể cộng một tensor float và một tensor integer, và bạn thậm chí không thể cộng một float 32-bit và một float 64-bit:

```
>>> tf.constant(2.) + tf.constant(40)
[...]
InvalidArgumentError: [...] expected to be a float tensor [...]

>>> tf.constant(2.) + tf.constant(40., dtype=tf.float64)
[...]
InvalidArgumentError: [...] expected to be a float tensor [...]
```

Điều này có thể hơi khó chịu lúc đầu, nhưng hãy nhớ rằng đó là vì một lý do chính đáng! Và tất nhiên bạn có thể sử dụng

`tf.cast()` khi bạn thực sự cần chuyển đổi kiểu:

```
>>> t2 = tf.constant(40., dtype=tf.float64)

>>> tf.constant(2.0) + tf.cast(t2, tf.float32)

<tf.Tensor: id=136, shape=(), dtype=float32, numpy=42.0>
```

Biến (Variables)

Các giá trị `tf.Tensor` mà chúng ta đã thấy cho đến nay là bất biến: chúng ta không thể sửa đổi chúng. Điều này có nghĩa là chúng ta không thể sử dụng các tensor thông thường để triển khai trọng số trong mạng nơ-ron, vì chúng cần được điều chỉnh bởi backpropagation. Thêm

vào đó, các tham số khác cũng có thể cần thay đổi theo thời gian (ví dụ: một bộ tối ưu hóa đã theo dõi các gradient trong quá khứ).

Những gì chúng ta cần là một `tf.Variable`:

```
>>> v = tf.Variable([[1., 2., 3.], [4., 5., 6.]])

>>> v
<tf.Variable 'Variable:0' shape=(2, 3) dtype=float32, numpy=
array([[1., 2., 3.],
       [4., 5., 6.]], dtype=float32)>
```

Một `tf.Variable` hoạt động rất giống một `tf.Tensor`: bạn có thể thực hiện các phép toán tương tự với nó, nó cũng tương thích tốt với NumPy, và nó cũng kén chọn về kiểu dữ liệu. Nhưng nó cũng có thể được sửa đổi tại chỗ bằng cách sử dụng phương thức `assign()` (hoặc `assign_add()` hoặc `assign_sub()`, cái này tăng hoặc giảm biến theo giá trị đã cho). Bạn cũng có thể sửa đổi từng ô (hoặc lát cắt), bằng cách sử dụng phương thức `assign()` của ô (hoặc lát cắt) hoặc bằng cách sử dụng các phương thức `scatter_update()` hoặc `scatter_nd_update()`:

```
v.assign(2 * v)           # v bây giờ bằng [[2., 4., 6.], [8., 10., 12.]]
v[0, 1].assign(42)        # v bây giờ bằng [[2., 42., 6.], [8., 10., 12.]]
v[:, 2].assign([0., 1.])  # v bây giờ bằng [[2., 42., 0.], [8., 10., 1.]]
v.scatter_nd_update(      # v bây giờ bằng [[100., 42., 0.], [8., 10., 200.]]
    indices=[[0, 0], [1, 2]], updates=[100., 200.]
```

Gán trực tiếp sẽ không hoạt động:

```
>>> v[1] = [7., 8., 9.]
[...]
TypeError: 'ResourceVariable' object does not support item assignment
```

Các cấu trúc dữ liệu khác

TensorFlow hỗ trợ một số cấu trúc dữ liệu khác, bao gồm những điều sau (xem phần “Các cấu trúc dữ liệu khác” trong sổ tay chương này hoặc Phụ lục C để biết thêm chi tiết):

- **Tensor thưa thớt (`tf.SparseTensor`):** Biểu diễn hiệu quả các tensor chứa chủ yếu là số 0. Gói `tf.sparse` chứa các phép toán cho tensor thưa thớt.
- **Mảng tensor (`tf.TensorArray`):** Là danh sách các tensor. Chúng có độ dài cố định theo mặc định nhưng có thể tùy chọn được mở rộng. Tất cả các tensor mà chúng chứa phải có cùng hình dạng và kiểu dữ liệu.
- **Tensor không đều (`tf.RaggedTensor`):** Biểu diễn danh sách các tensor, tất cả đều cùng bậc và kiểu dữ liệu, nhưng có kích thước thay đổi. Các chiều mà kích thước tensor thay đổi được gọi là các chiều không đều. Gói `tf.ragged` chứa các phép toán cho tensor không đều.

- **Tensor chuỗi:** Là các tensor thông thường có kiểu `tf.string`. Chúng đại diện cho các chuỗi byte, không phải chuỗi Unicode, vì vậy nếu bạn tạo một tensor chuỗi bằng cách sử dụng một chuỗi Unicode (ví dụ: một chuỗi Python 3 thông thường như “café”), thì nó sẽ được mã hóa sang UTF-8 tự động (ví dụ: `b"caf\xc3\xa9"`). Ngoài ra, bạn có thể biểu diễn chuỗi Unicode bằng cách sử dụng các tensor kiểu `tf.int32`, trong đó mỗi mục đại diện cho một điểm mã Unicode (ví dụ: `[99, 97, 102, 233]`). Gói `tf.strings` (có `s`) chứa các phép toán cho chuỗi byte và chuỗi Unicode (và để chuyển đổi cái này sang cái kia). Điều quan trọng cần lưu ý là một `tf.string` là nguyên tử, nghĩa là độ dài của nó không xuất hiện trong hình dạng của tensor. Một khi bạn chuyển đổi nó thành một tensor Unicode (tức là một tensor kiểu `tf.int32` chứa các điểm mã Unicode), độ dài sẽ xuất hiện trong hình dạng.
- **Tập hợp:** Được biểu diễn dưới dạng các tensor thông thường (hoặc tensor thưa thớt). Ví dụ, `tf.constant([[1, 2], [3, 4]])` đại diện cho hai tập hợp {1, 2} và {3, 4}. Tổng quát hơn, mỗi tập hợp được biểu diễn bởi một vector trong trục cuối cùng của tensor. Bạn có thể thao tác với các tập hợp bằng cách sử dụng các phép toán từ gói `tf.sets`.
- **Hàng đợi:** Lưu trữ các tensor qua nhiều bước. TensorFlow cung cấp nhiều loại hàng đợi khác nhau: hàng đợi FIFO cơ bản (`FIFOQueue`), cộng với các hàng đợi có thể ưu tiên một số mục (`PriorityQueue`), xáo trộn các mục của chúng (`RandomShuffleQueue`), và nhóm các mục có hình dạng khác nhau bằng cách đệm (`PaddingFIFOQueue`). Các lớp này đều nằm trong gói `tf.queue`.

Với các tensor, phép toán, biến và các cấu trúc dữ liệu khác nhau theo ý của bạn, bây giờ bạn đã sẵn sàng tùy chỉnh các mô hình và thuật toán huấn luyện của mình!

Tùy chỉnh mô hình và thuật toán huấn luyện

Bạn sẽ bắt đầu bằng cách tạo một hàm mất mát tùy chỉnh, đây là một trường hợp sử dụng đơn giản và phổ biến.

Các hàm mất mát tùy chỉnh

Giả sử bạn muốn huấn luyện một mô hình hồi quy, nhưng tập huấn luyện của bạn hơi nhiễu. Tất nhiên, bạn bắt đầu bằng cách cố gắng làm sạch tập dữ liệu của mình bằng cách loại bỏ hoặc sửa các giá trị ngoại lai, nhưng điều đó hóa ra không đủ; tập dữ liệu vẫn bị nhiễu. Bạn nên sử dụng hàm mất mát nào? Sai số bình phương trung bình có thể phạt các lỗi lớn quá nhiều và khiến mô hình của bạn không chính xác. Sai số tuyệt đối trung bình sẽ không phạt các giá trị ngoại lai nhiều, nhưng quá trình huấn luyện có thể mất một thời gian để hội tụ, và mô hình đã huấn luyện có thể không chính xác lắm. Đây có lẽ là thời điểm tốt để sử dụng mất mát Huber (được giới thiệu trong Chương 10) thay vì MSE cũ kỹ. Mất mát Huber có sẵn trong Keras (chỉ cần sử dụng một thể hiện của lớp

`tf.keras.losses.Huber`), nhưng hãy giả vờ nó không có ở đó. Để triển khai nó, chỉ cần tạo một hàm nhận các nhãn và dự đoán của mô hình làm đối số, và sử dụng các phép toán TensorFlow để tính toán một tensor chứa tất cả các mất mát (một cho mỗi mẫu):


```
def huber_fn(y_true, y_pred):
    error = y_true - y_pred
    is_small_error = tf.abs(error) < 1
    squared_loss = tf.square(error) / 2
    linear_loss = tf.abs(error) - 0.5
    return tf.where(is_small_error, squared_loss, linear_loss)
```

Cũng có thể trả về mất mát trung bình thay vì các mất mát mẫu riêng lẻ, nhưng điều này không được khuyến khích vì nó khiến không thể sử dụng trọng số lớp hoặc trọng số mẫu khi bạn cần chúng (xem Chương 10).

Bây giờ bạn có thể sử dụng hàm mất mát Huber này khi bạn biên dịch mô hình Keras, sau đó huấn luyện mô hình của bạn như bình thường:

```
model.compile(loss=huber_fn, optimizer="nadam")
model.fit(X_train, y_train, [...])
```

Và thế là xong! Đối với mỗi batch trong quá trình huấn luyện, Keras sẽ gọi hàm

`huber_fn()` để tính toán mất mát, sau đó nó sẽ sử dụng tự động đạo hàm ngược để tính toán gradient của mất mát đối với tất cả các tham số mô hình, và cuối cùng nó sẽ thực hiện một bước giảm độ dốc (trong ví dụ này sử dụng bộ tối ưu hóa Nadam). Hơn nữa, nó sẽ theo dõi tổng mất mát kể từ đầu epoch, và nó sẽ hiển thị mất mát trung bình.

Nhưng điều gì xảy ra với mất mát tùy chỉnh này khi bạn lưu mô hình?

Lưu và tải mô hình chứa các thành phần tùy chỉnh

Lưu một mô hình chứa hàm mất mát tùy chỉnh hoạt động tốt, nhưng khi bạn tải nó, bạn sẽ cần cung cấp một từ điển ánh xạ tên hàm tới hàm thực tế. Tổng quát hơn, khi bạn tải một mô hình chứa các đối tượng tùy chỉnh, bạn cần ánh xạ các tên tới các đối tượng:

```
model = tf.keras.models.load_model("my_model_with_a_custom_loss",
                                   custom_objects={"huber_fn": huber_fn})
```

Với việc triển khai hiện tại, bất kỳ lỗi nào giữa -1 và 1 đều được coi là “nhỏ”. Nhưng nếu bạn muốn một ngưỡng khác thì sao? Một giải pháp là tạo một hàm tạo một hàm mất mát đã được cấu hình:

```
def create_huber(threshold=1.0):
    def huber_fn(y_true, y_pred):
        error = y_true - y_pred
        is_small_error = tf.abs(error) < threshold
        squared_loss = tf.square(error) / 2
        linear_loss = threshold * tf.abs(error) - threshold ** 2 / 2
        return tf.where(is_small_error, squared_loss, linear_loss)
    return huber_fn

model.compile(loss=create_huber(2.0), optimizer="nadam")
```

Thật không may, khi bạn lưu mô hình, ngưỡng sẽ không được lưu. Điều này có nghĩa là bạn sẽ phải chỉ định giá trị ngưỡng khi tải mô hình (lưu ý rằng tên cần sử dụng là “huber_fn”, là tên của hàm bạn đã cung cấp cho Keras, không phải tên của hàm đã tạo ra nó):

```
model = tf.keras.models.load_model(
    "my_model_with_a_custom_loss_threshold_2",
    custom_objects={"huber_fn": create_huber(2.0)}
)
```

Bạn có thể giải quyết vấn đề này bằng cách tạo một lớp con của lớp

`tf.keras.losses.Loss`, và sau đó triển khai phương thức `get_config()` của nó:

```
class HuberLoss(tf.keras.losses.Loss):
    def __init__(self, threshold=1.0, **kwargs):
        self.threshold = threshold
        super().__init__(**kwargs)

    def call(self, y_true, y_pred):
        error = y_true - y_pred
        is_small_error = tf.abs(error) < self.threshold
        squared_loss = tf.square(error) / 2
        linear_loss = self.threshold * tf.abs(error) - self.threshold ** 2 /
2
        return tf.where(is_small_error, squared_loss, linear_loss)

    def get_config(self):
        base_config = super().get_config()
        return {**base_config, "threshold": self.threshold}
```

Hãy cùng xem đoạn mã này:

- Hàm tạo chấp nhận `**kwargs` và truyền chúng cho hàm tạo cha, cái này xử lý các siêu tham số tiêu chuẩn: tên của mất mát và thuật toán giảm thiểu được sử dụng để tổng hợp các mất mát thể hiện riêng lẻ. Theo mặc định, đây là “AUTO”, tương đương với “SUM_OVER_BATCH_SIZE”: mất mát sẽ là tổng các mất mát thể hiện, được trọng số hóa bằng trọng số mẫu (nếu có), và chia cho kích thước batch (không phải tổng trọng số, vì vậy đây không phải là trung bình có trọng số). Các giá trị có thể có khác là “SUM” và “NONE”.
- Phương thức `call()` nhận các nhãn và dự đoán, tính toán tất cả các mất mát thể hiện và trả về chúng.
- Phương thức `get_config()` trả về một từ điển ánh xạ tên siêu tham số tới giá trị của nó. Nó trước tiên gọi phương thức `get_config()` của lớp cha, sau đó thêm các siêu tham số mới vào từ điển này.

Bạn có thể sử dụng bất kỳ thể hiện nào của lớp này khi bạn biên dịch mô hình:

```
model.compile(loss=HuberLoss(2.), optimizer="nadam")
```

Khi bạn lưu mô hình, ngưỡng sẽ được lưu cùng với nó; và khi bạn tải mô hình, bạn chỉ cần ánh xạ tên lớp tới chính lớp đó:

```
model = tf.keras.models.load_model("my_model_with_a_custom_loss_class",
                                   custom_objects={"HuberLoss": HuberLoss})
```

Khi bạn lưu một mô hình, Keras gọi phương thức

`get_config()` của thể hiện mất mát và lưu cấu hình ở định dạng SavedModel. Khi bạn tải mô hình, nó gọi phương thức lớp

`from_config()` trên lớp HuberLoss: phương thức này được triển khai bởi lớp cơ sở (Loss) và tạo một thể hiện của lớp, truyền `**config` vào hàm tạo.

Đó là tất cả về mất mát! Như bạn sẽ thấy bây giờ, các hàm kích hoạt, trình khởi tạo, trình chính quy hóa và ràng buộc tùy chỉnh không khác nhiều.

Các hàm kích hoạt, trình khởi tạo, trình chính quy hóa và ràng buộc tùy chỉnh

Hầu hết các chức năng của Keras, chẳng hạn như hàm mất mát, trình chính quy hóa, ràng buộc, trình khởi tạo, số liệu, hàm kích hoạt, lớp, và thậm chí cả mô hình đầy đủ, có thể được tùy chỉnh theo cách tương tự. Hầu hết thời gian, bạn chỉ cần viết một hàm đơn giản với các đầu vào và đầu ra thích hợp. Dưới đây là các ví dụ về một hàm kích hoạt tùy chỉnh (tương đương với `tf.keras.activations.softplus()` hoặc `tf.nn.softplus()`), một trình khởi tạo Glorot tùy chỉnh (tương đương với `tf.keras.initializers.glorot_normal()`), một trình chính quy hóa l_1 tùy chỉnh (tương đương với `tf.keras.regularizers.l1(0.01)`), và một ràng buộc tùy chỉnh đảm bảo tất cả các trọng số đều dương (tương đương với `tf.keras.constraints.nonneg()` hoặc `tf.nn.relu()`):

```
def my_softplus(z):
    return tf.math.log(1.0 + tf.exp(z))

def my_glorot_initializer(shape, dtype=tf.float32):
    stddev = tf.sqrt(2. / (shape[0] + shape[1]))
    return tf.random.normal(shape, stddev=stddev, dtype=dtype)

def my_l1_regularizer(weights):
    return tf.reduce_sum(tf.abs(0.01 * weights))

def my_positive_weights(weights): # giá trị trả về chỉ là tf.nn.relu(weights)
    return tf.where(weights < 0., tf.zeros_like(weights), weights)
```

Như bạn có thể thấy, các đối số phụ thuộc vào loại hàm tùy chỉnh. Các hàm tùy chỉnh này sau đó có thể được sử dụng bình thường, như được thể hiện ở đây:

```
layer = tf.keras.layers.Dense(1, activation=my_softplus,
                               kernel_initializer=my_glorot_initializer,
                               kernel_regularizer=my_l1_regularizer,
                               kernel_constraint=my_positive_weights)
```

Hàm kích hoạt sẽ được áp dụng cho đầu ra của lớp Dense này, và kết quả của nó sẽ được chuyển đến lớp tiếp theo. Trọng số của lớp sẽ được khởi tạo bằng giá trị trả về của trình khởi tạo. Ở mỗi bước huấn luyện, trọng số sẽ được truyền cho hàm chính quy hóa để tính toán mất mát chính quy hóa, mất mát này sẽ được thêm vào mất mát chính để có được mất mát cuối cùng được sử dụng cho quá trình huấn luyện. Cuối cùng, hàm ràng buộc sẽ được gọi sau mỗi bước huấn luyện, và trọng số của lớp sẽ được thay thế bằng trọng số bị ràng buộc.

Nếu một hàm có các siêu tham số cần được lưu cùng với mô hình, thì bạn sẽ muốn tạo lớp con từ lớp thích hợp, chẳng hạn như `tf.keras.regularizers.Regularizer`, `tf.keras.constraints.Constraint`, `tf.keras.initializers.Initializer`, hoặc `tf.keras.layers.Layer` (đối với bất kỳ lớp nào, bao gồm cả hàm kích hoạt). Tương tự như cách bạn đã làm với hàm mất mát tùy chỉnh, đây là một lớp đơn giản cho chính quy hóa l_1 lưu siêu tham số factor của nó (lần này bạn không cần gọi hàm tạo cha hoặc phương thức `get_config()`, vì chúng không được định nghĩa bởi lớp cha):

```
class MyL1Regularizer(tf.keras.regularizers.Regularizer):
    def __init__(self, factor):
        self.factor = factor

    def __call__(self, weights):
        return tf.reduce_sum(tf.abs(self.factor * weights))

    def get_config(self):
        return {"factor": self.factor}
```

Lưu ý rằng bạn phải triển khai phương thức `call()` cho các hàm mất mát, lớp (bao gồm cả hàm kích hoạt), và mô hình, hoặc phương thức `__call__()` cho các trình chính quy hóa, trình khởi tạo và ràng buộc. Đối với các số liệu, mọi thứ hơi khác một chút, như bạn sẽ thấy bây giờ.

Các số liệu tùy chỉnh

Các hàm mất mát và số liệu về mặt khái niệm không giống nhau: các hàm mất mát (ví dụ: entropy chéo) được giảm độ dốc sử dụng để huấn luyện một mô hình, vì vậy chúng phải khả vi (ít nhất là tại các điểm chúng được đánh giá), và gradient của chúng không nên bằng 0 ở khắp mọi nơi. Hơn nữa, chúng không cần phải dễ hiểu đối với con người. Ngược lại, các số liệu (ví dụ: độ chính xác) được sử dụng để đánh giá một mô hình: chúng phải dễ hiểu hơn, và chúng có thể không khả vi hoặc có gradient bằng 0 ở khắp mọi nơi.

Điều đó nói lên rằng, trong hầu hết các trường hợp, việc định nghĩa một hàm số liệu tùy chỉnh hoàn toàn giống như định nghĩa một hàm mất mát tùy chỉnh. Thực tế, chúng ta thậm chí có thể sử dụng hàm mất mát Huber mà chúng ta đã tạo trước đó làm một số liệu; nó sẽ hoạt động tốt (và tính bền vững cũng sẽ hoạt động theo cùng một cách, trong trường hợp này chỉ lưu tên của hàm, “`huber_fn`”, không phải ngưỡng):

```
model.compile(loss="mse", optimizer="nadam", metrics=[create_huber(2.0)])
```

Đối với mỗi batch trong quá trình huấn luyện, Keras sẽ tính toán số liệu này và theo dõi trung bình của nó kể từ đầu epoch. Hầu hết thời gian, đây chính xác là những gì bạn muốn. Nhưng không phải lúc nào cũng vậy! Hãy xem xét độ chính xác của một bộ phân loại nhị phân, chẳng hạn. Như bạn đã thấy trong Chương 3, độ chính xác là số lượng dương tính đúng chia cho số lượng dự đoán dương tính (bao gồm cả dương tính đúng và dương tính sai). Giả sử mô hình đã thực hiện năm dự đoán dương tính trong batch đầu tiên, bốn trong số đó là đúng: đó là độ chính xác 80%. Sau đó, giả sử mô hình đã thực hiện ba dự đoán dương tính trong batch thứ hai, nhưng tất cả đều sai: đó là độ chính xác 0% cho batch thứ hai. Nếu bạn chỉ tính trung bình của hai độ chính xác này, bạn sẽ nhận được 40%. Nhưng khoan đã — đó không phải là độ chính xác của mô hình trên hai batch này! Thật vậy, có tổng cộng bốn dương tính đúng (4 + 0) trong số tám dự đoán dương tính (5 + 3), vì vậy độ chính xác tổng thể là 50%, không phải 40%. Những gì chúng ta cần là một đối tượng có thể theo dõi số lượng dương tính đúng và số lượng dương tính sai và có thể tính toán độ chính xác dựa trên các số này khi được yêu cầu. Đây chính xác là những gì lớp `tf.keras.metrics.Precision` thực hiện:

```
>>> precision = tf.keras.metrics.Precision()

>>> precision([0, 1, 1, 1, 0, 1, 0, 1], [1, 1, 0, 1, 0, 1, 0, 1])
<tf.Tensor: shape=(), dtype=float32, numpy=0.8>

>>> precision([0, 1, 0, 0, 1, 0, 1, 1], [1, 0, 1, 1, 0, 0, 0, 0])
<tf.Tensor: shape=(), dtype=float32, numpy=0.5>
```

Trong ví dụ này, chúng ta đã tạo một đối tượng `Precision`, sau đó chúng ta đã sử dụng nó như một hàm, truyền cho nó các nhãn và dự đoán cho batch đầu tiên, sau đó cho batch thứ hai (bạn có thể tùy chọn truyền trọng số mẫu nếu muốn). Chúng ta đã sử dụng cùng số lượng dương tính đúng và dương tính sai như trong ví dụ chúng ta vừa thảo luận. Sau batch đầu tiên, nó trả về độ chính xác 80%; sau đó sau batch thứ hai, nó trả về 50% (là độ chính xác tổng thể cho đến nay, không phải độ chính xác của batch thứ hai). Đây được gọi là **số liệu theo luồng** (hoặc số liệu trạng thái), vì nó được cập nhật dần dần, từng batch một.

Tại bất kỳ thời điểm nào, chúng ta có thể gọi phương thức `result()` để lấy giá trị hiện tại của số liệu. Chúng ta cũng có thể xem các biến của nó (theo dõi số lượng dương tính đúng và dương tính sai) bằng cách sử dụng thuộc tính `variables`, và chúng ta có thể đặt lại các biến này bằng phương thức `reset_states()`:

```
>>> precision.result()
<tf.Tensor: shape=(), dtype=float32, numpy=0.5>

>>> precision.variables
[<tf.Variable 'true_positives:0' [...], numpy=array([4.], dtype=float32)>,
 <tf.Variable 'false_positives:0' [...], numpy=array([4.], dtype=float32)>]

>>> precision.reset_states() # cả hai biến đều được đặt lại về 0.0
```

Nếu bạn cần định nghĩa số liệu theo luồng tùy chỉnh của riêng mình, hãy tạo một lớp con của lớp `tf.keras.metrics.Metric`. Dưới đây là một ví dụ cơ bản theo dõi tổng mất mát Huber

và số lượng thể hiện đã thấy cho đến nay. Khi được yêu cầu kết quả, nó trả về tỷ lệ, đó chính là trung bình mất mát Huber:

```
class HuberMetric(tf.keras.metrics.Metric):
    def __init__(self, threshold=1.0, **kwargs):
        super().__init__(**kwargs) # xử lý các đối số cơ sở (ví dụ: dtype)
        self.threshold = threshold
        self.huber_fn = create_huber(threshold)
        self.total = self.add_weight("total", initializer="zeros")
        self.count = self.add_weight("count", initializer="zeros")

    def update_state(self, y_true, y_pred, sample_weight=None):
        sample_metrics = self.huber_fn(y_true, y_pred)
        self.total.assign_add(tf.reduce_sum(sample_metrics))
        self.count.assign_add(tf.cast(tf.size(y_true), tf.float32))

    def result(self):
        return self.total / self.count

    def get_config(self):
        base_config = super().get_config()
        return {**base_config, "threshold": self.threshold}
```

Hãy cùng xem đoạn mã này:

- Hàm tạo sử dụng phương thức `add_weight()` để tạo các biến cần thiết để theo dõi trạng thái của số liệu qua nhiều batch — trong trường hợp này, tổng của tất cả các mất mát Huber (`total`) và số lượng thể hiện đã thấy cho đến nay (`count`). Bạn có thể chỉ cần tạo các biến thủ công nếu bạn thích. Keras theo dõi bất kỳ `tf.Variable` nào được đặt làm thuộc tính (và tổng quát hơn, bất kỳ đối tượng “có thể theo dõi” nào, chẳng hạn như lớp hoặc mô hình).
- Phương thức `update_state()` được gọi khi bạn sử dụng một thể hiện của lớp này làm hàm (như chúng ta đã làm với đối tượng `Precision`). Nó cập nhật các biến, đưa ra các nhãn và dự đoán cho một batch (và trọng số mẫu, nhưng trong trường hợp này chúng ta bỏ qua chúng).
- Phương thức `result()` tính toán và trả về kết quả cuối cùng, trong trường hợp này là số liệu Huber trung bình trên tất cả các thể hiện. Khi bạn sử dụng số liệu làm hàm, phương thức `update_state()` được gọi trước, sau đó phương thức `result()` được gọi, và đầu ra của nó được trả về.
- Chúng ta cũng triển khai phương thức `get_config()` để đảm bảo ngưỡng được lưu cùng với mô hình.
- Việc triển khai mặc định của phương thức `reset_states()` đặt lại tất cả các biến về 0.0 (nhưng bạn có thể ghi đè nó nếu cần).

Khi bạn định nghĩa một số liệu bằng một hàm đơn giản, Keras tự động gọi nó cho mỗi batch, và nó theo dõi trung bình trong mỗi epoch, giống như chúng ta đã làm thủ công. Vì vậy, lợi ích

duy nhất của lớp HuberMetric của chúng ta là ngưỡng sẽ được lưu. Nhưng tất nhiên, một số số liệu, như độ chính xác, không thể đơn giản là tính trung bình trên các batch: trong những trường hợp đó, không có lựa chọn nào khác ngoài việc triển khai một số liệu theo luồng.

Bây giờ bạn đã xây dựng một số liệu theo luồng, việc xây dựng một lớp tùy chỉnh sẽ trở nên dễ dàng!

Các lớp tùy chỉnh

Bạn có thể thỉnh thoảng muốn xây dựng một kiến trúc chứa một lớp kỳ lạ mà TensorFlow không cung cấp triển khai mặc định. Hoặc bạn có thể đơn giản là muốn xây dựng một kiến trúc rất lặp đi lặp lại, trong đó một khối lớp cụ thể được lặp lại nhiều lần, và sẽ thuận tiện khi coi mỗi khối là một lớp duy nhất. Đối với những trường hợp như vậy, bạn sẽ muốn xây dựng một lớp tùy chỉnh.

Có một số lớp không có trọng số, chẳng hạn như `tf.keras.layers.Flatten` hoặc `tf.keras.layers.ReLU`. Nếu bạn muốn tạo một lớp tùy chỉnh không có bất kỳ trọng số nào, lựa chọn đơn giản nhất là viết một hàm và bọc nó trong một lớp `tf.keras.layers.Lambda`. Ví dụ, lớp sau đây sẽ áp dụng hàm mũ cho các đầu vào của nó:

```
exponential_layer = tf.keras.layers.Lambda(lambda x: tf.exp(x))
```

Lớp tùy chỉnh này sau đó có thể được sử dụng như bất kỳ lớp nào khác, sử dụng API tuần tự, API chức năng hoặc API subclassing. Bạn cũng có thể sử dụng nó làm hàm kích hoạt, hoặc bạn có thể sử dụng `activation=tf.exp`. Lớp mũ đôi khi được sử dụng trong lớp đầu ra của một mô hình hồi quy khi các giá trị cần dự đoán có các thang đo rất khác nhau (ví dụ: 0.001, 10., 1.000.). Thực tế, hàm mũ là một trong những hàm kích hoạt tiêu chuẩn trong Keras, vì vậy bạn có thể chỉ cần sử dụng `activation="exponential"`.

Như bạn có thể đoán, để xây dựng một lớp trạng thái tùy chỉnh (tức là một lớp có trọng số), bạn cần tạo một lớp con của lớp `tf.keras.layers.Layer`. Ví dụ, lớp sau đây triển khai một phiên bản đơn giản hóa của lớp Dense:

```
class MyDense(tf.keras.layers.Layer):
    def __init__(self, units, activation=None, **kwargs):
        super().__init__(**kwargs)
        self.units = units
        self.activation = tf.keras.activations.get(activation)

    def build(self, batch_input_shape):
        self.kernel = self.add_weight(
            name="kernel", shape=[batch_input_shape[-1], self.units],
            initializer="glorot_normal")
        self.bias = self.add_weight(
            name="bias", shape=[self.units], initializer="zeros")

    def call(self, X):
        return self.activation(X @ self.kernel + self.bias)
```

```
def get_config(self):
    base_config = super().get_config()
    return {**base_config, "units": self.units, "activation":
            tf.keras.activations.serialize(self.activation)}
```

Hãy cùng xem đoạn mã này:

- Hàm tạo nhận tất cả các siêu tham số làm đối số (trong ví dụ này, units và activation), và quan trọng là nó cũng nhận một đối số `**kwargs`. Nó gọi hàm tạo cha, truyền các kwargs cho nó: điều này xử lý các đối số tiêu chuẩn như `input_shape`, `trainable`, và `name`. Sau đó, nó lưu các siêu tham số dưới dạng thuộc tính, chuyển đổi đối số activation thành hàm kích hoạt thích hợp bằng cách sử dụng hàm `tf.keras.activations.get()` (nó chấp nhận các hàm, các chuỗi tiêu chuẩn như “relu” hoặc “swish”, hoặc đơn giản là None).
- Vai trò của phương thức `build()` là tạo các biến của lớp bằng cách gọi phương thức `add_weight()` cho mỗi trọng số. Phương thức `build()` được gọi lần đầu tiên khi lớp được sử dụng. Tại thời điểm đó, Keras sẽ biết hình dạng của đầu vào của lớp này, và nó sẽ truyền hình dạng đó cho phương thức `build()`, điều này thường cần thiết để tạo ra một số trọng số. Ví dụ, chúng ta cần biết số lượng nơ-ron trong lớp trước đó để tạo ma trận trọng số kết nối (tức là “kernel”): điều này tương ứng với kích thước của chiều cuối cùng của đầu vào. Ở cuối phương thức `build()` (và chỉ ở cuối), bạn phải gọi phương thức `build()` của lớp cha: điều này cho Keras biết rằng lớp đã được xây dựng (nó chỉ đặt `self.built = True`).
- Phương thức `call()` thực hiện các phép toán mong muốn. Trong trường hợp này, chúng ta tính toán phép nhân ma trận giữa các đầu vào X và kernel của lớp, chúng ta cộng vector bias, và chúng ta áp dụng hàm kích hoạt cho kết quả, và điều này cho chúng ta đầu ra của lớp.
- Phương thức `get_config()` giống như trong các lớp tùy chỉnh trước đó. Lưu ý rằng chúng ta lưu cấu hình đầy đủ của hàm kích hoạt bằng cách gọi `tf.keras.activations.serialize()`.

Bạn có thể sử dụng lớp `MyDense` giống như bất kỳ lớp nào khác!

Để tạo một lớp có nhiều đầu vào (ví dụ: `Concatenate`), đối số cho phương thức `call()` phải là một tuple chứa tất cả các đầu vào. Để tạo một lớp có nhiều đầu ra, phương thức `call()` nên trả về danh sách các đầu ra. Ví dụ, lớp đồ chơi sau đây nhận hai đầu vào và trả về ba đầu ra:

```
class MyMultiLayer(tf.keras.layers.Layer):
    def call(self, X):
        X1, X2 = X
        return X1 + X2, X1 * X2, X1 / X2
```

Lớp này bây giờ có thể được sử dụng như bất kỳ lớp nào khác, nhưng tất nhiên chỉ sử dụng API chức năng và API subclassing, không phải API tuần tự (chỉ chấp nhận các lớp có một đầu vào và một đầu ra).

Nếu lớp của bạn cần có hành vi khác nhau trong quá trình huấn luyện và trong quá trình kiểm tra (ví dụ: nếu nó sử dụng các lớp Dropout hoặc BatchNormalization), thì bạn phải thêm đối số `training` vào phương thức `call()` và sử dụng đối số này để quyết định phải làm gì. Ví dụ, hãy tạo một lớp thêm nhiễu Gaussian trong quá trình huấn luyện (để chính quy hóa) nhưng không làm gì trong quá trình kiểm tra (Keras có một lớp làm điều tương tự, `tf.keras.layers.GaussianNoise`):

```
class MyGaussianNoise(tf.keras.layers.Layer):
    def __init__(self, stddev, **kwargs):
        super().__init__(**kwargs)
        self.stddev = stddev

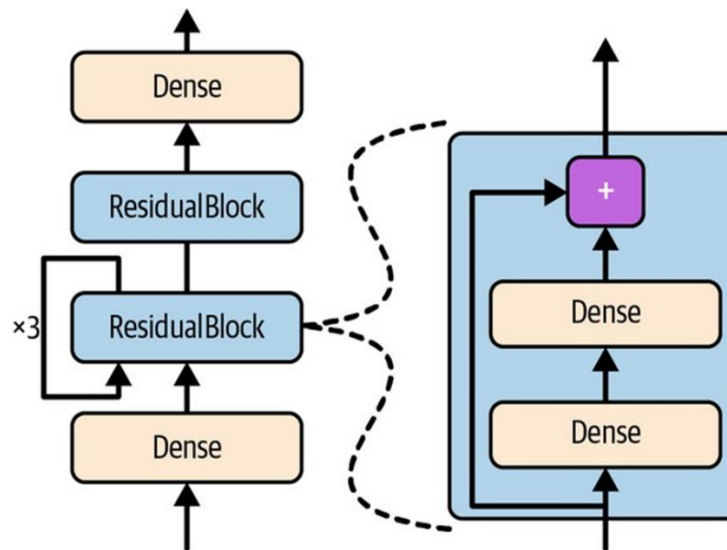
    def call(self, X, training=False):
        if training:
            noise = tf.random.normal(tf.shape(X), stddev=self.stddev)
            return X + noise
        else:
            return X
```

Với điều đó, bây giờ bạn có thể xây dựng bất kỳ lớp tùy chỉnh nào bạn cần! Bây giờ chúng ta hãy xem cách tạo các mô hình tùy chỉnh.

Mô hình tùy chỉnh

Chúng ta đã xem xét việc tạo các lớp mô hình tùy chỉnh trong Chương 10, khi chúng ta thảo luận về API subclassing. Thật đơn giản: kế thừa từ lớp

`tf.keras.Model`, tạo các lớp và biến trong hàm tạo, và triển khai phương thức `call()` để làm bất cứ điều gì bạn muốn mô hình thực hiện. Ví dụ, giả sử chúng ta muốn xây dựng mô hình được thể hiện trong Hình 12-3.



Hình 12-3. Ví dụ mô hình tùy chỉnh: một mô hình tùy ý với lớp ResidualBlock tùy chỉnh chứa kết nối bỏ qua

Các đầu vào đi qua một lớp dày đặc đầu tiên, sau đó qua một khối residual bao gồm hai lớp dày đặc và một phép toán cộng (như bạn sẽ thấy trong Chương 14, một khối residual cộng đầu vào của nó vào đầu ra của nó), sau đó qua cùng khối residual này thêm ba lần nữa, sau đó qua một khối residual thứ hai, và kết quả cuối cùng đi qua một lớp đầu ra dày đặc. Đừng lo lắng nếu mô hình này không có nhiều ý nghĩa; đó chỉ là một ví dụ để minh họa thực tế rằng bạn có thể dễ dàng xây dựng bất kỳ loại mô hình nào bạn muốn, ngay cả một mô hình chứa các vòng lặp và kết nối bỏ qua. Để triển khai mô hình này, tốt nhất là tạo một lớp ResidualBlock trước, vì chúng ta sẽ tạo một vài khối giống hệt nhau (và chúng ta có thể muốn tái sử dụng nó trong một mô hình khác):

```
class ResidualBlock(tf.keras.layers.Layer):
    def __init__(self, n_layers, n_neurons, **kwargs):
        super().__init__(**kwargs)
        self.hidden = [tf.keras.layers.Dense(n_neurons, activation="relu",
                                                kernel_initializer="he_normal")
                        for _ in range(n_layers)]

    def call(self, inputs):
        Z = inputs
        for layer in self.hidden:
            Z = layer(Z)
        return inputs + Z
```

Lớp này hơi đặc biệt vì nó chứa các lớp khác. Điều này được Keras xử lý một cách minh bạch: nó tự động phát hiện rằng thuộc tính hidden chứa các đối tượng có thể theo dõi (các lớp trong trường hợp này), vì vậy các biến của chúng tự động được thêm vào danh sách các biến của lớp này. Phần còn lại của lớp này tự giải thích. Tiếp theo, hãy sử dụng API subclassing để định nghĩa mô hình đó:

```

class ResidualRegressor(tf.keras.Model):
    def __init__(self, output_dim, **kwargs):
        super().__init__(**kwargs)
        self.hidden1 = tf.keras.layers.Dense(30, activation="relu",
                                                kernel_initializer="he_normal")

        self.block1 = ResidualBlock(2, 30)
        self.block2 = ResidualBlock(2, 30)
        self.out = tf.keras.layers.Dense(output_dim)

    def call(self, inputs):
        Z = self.hidden1(inputs)
        for _ in range(1 + 3):
            Z = self.block1(Z)
        Z = self.block2(Z)
        return self.out(Z)

```

Chúng ta tạo các lớp trong hàm tạo và sử dụng chúng trong phương thức `call()`. Mô hình này sau đó có thể được sử dụng như bất kỳ mô hình nào khác (biên dịch nó, huấn luyện nó, đánh giá nó và sử dụng nó để đưa ra dự đoán). Nếu bạn cũng muốn có thể lưu mô hình bằng phương thức `save()` và tải nó bằng hàm `tf.keras.models.load_model()`, bạn phải triển khai phương thức `get_config()` (như chúng ta đã làm trước đó) trong cả lớp `ResidualBlock` và lớp `ResidualRegressor`. Ngoài ra, bạn có thể lưu và tải trọng số bằng các phương thức `save_weights()` và `load_weights()`.

Lớp `Model` là một lớp con của lớp `Layer`, vì vậy các mô hình có thể được định nghĩa và sử dụng chính xác như các lớp. Nhưng một mô hình có một số chức năng bổ sung, bao gồm tất nhiên các phương thức `compile()`, `fit()`, `evaluate()`, và `predict()` (và một vài biến thể), cộng với phương thức `get_layer()` (có thể trả về bất kỳ lớp nào của mô hình theo tên hoặc theo chỉ số) và phương thức `save()` (và hỗ trợ cho `tf.keras.models.load_model()` và `tf.keras.models.clone_model()`).

Với điều đó, bạn có thể tự nhiên và ngắn gọn xây dựng gần như bất kỳ mô hình nào bạn tìm thấy trong một bài báo, sử dụng API tuần tự, API chức năng, API subclassing, hoặc thậm chí là sự kết hợp của chúng. “Gần như” bất kỳ mô hình nào?

Vâng, vẫn còn một vài điều chúng ta cần xem xét: thứ nhất, làm thế nào để định nghĩa các hàm mất mát hoặc số liệu dựa trên các chi tiết nội bộ của mô hình, và thứ hai, làm thế nào để xây dựng một vòng lặp huấn luyện tùy chỉnh.

Các hàm mất mát và số liệu dựa trên các chi tiết nội bộ của mô hình

Các hàm mất mát và số liệu tùy chỉnh mà chúng ta đã định nghĩa trước đó đều dựa trên các nhãn và dự đoán (và tùy chọn là trọng số mẫu).

Sẽ có những lúc bạn muốn định nghĩa các hàm mất mát dựa trên các phần khác của mô hình của bạn, chẳng hạn như trọng số hoặc kích hoạt của các lớp ẩn của nó. Điều này có thể hữu ích cho mục đích chính quy hóa hoặc để giám sát một số khía cạnh nội bộ của mô hình của bạn.

Để định nghĩa một hàm mất mát tùy chỉnh dựa trên các chi tiết nội bộ của mô hình, hãy tính toán nó dựa trên bất kỳ phần nào của mô hình bạn muốn, sau đó truyền kết quả cho phương thức `add_loss()`. Ví dụ, hãy xây dựng một mô hình MLP hồi quy tùy chỉnh bao gồm một chồng năm lớp ẩn cộng với một lớp đầu ra. Mô hình tùy chỉnh này cũng sẽ có một đầu ra phụ trợ trên lớp ẩn trên cùng. Mất mát liên quan đến đầu ra phụ trợ này sẽ được gọi là **mất mát tái tạo** (xem Chương 17): đó là sự khác biệt bình phương trung bình giữa tái tạo và đầu vào. Bằng cách thêm mất mát tái tạo này vào mất mát chính, chúng ta sẽ khuyến khích mô hình bảo toàn càng nhiều thông tin càng tốt thông qua các lớp ẩn — ngay cả thông tin không trực tiếp hữu ích cho chính tác vụ hồi quy. Trong thực tế, mất mát này đôi khi cải thiện khả năng tổng quát hóa (nó là một mất mát chính quy hóa). Cũng có thể thêm một số liệu tùy chỉnh bằng cách sử dụng phương thức `add_metric()` của mô hình. Dưới đây là mã cho mô hình tùy chỉnh này với một mất mát tái tạo tùy chỉnh và một số liệu tương ứng:

```
class ReconstructingRegressor(tf.keras.Model):
    def __init__(self, output_dim, **kwargs):
        super().__init__(**kwargs)
        self.hidden = [tf.keras.layers.Dense(30, activation="relu",
                                              kernel_initializer="he_normal")
                       for _ in range(5)]
        self.out = tf.keras.layers.Dense(output_dim)
        self.reconstruction_mean = tf.keras.metrics.Mean(
            name="reconstruction_error")

    def build(self, batch_input_shape):
        n_inputs = batch_input_shape[-1]
        self.reconstruct = tf.keras.layers.Dense(n_inputs)

    def call(self, inputs, training=False):
        Z = inputs
        for layer in self.hidden:
            Z = layer(Z)
        reconstruction = self.reconstruct(Z)
        recon_loss = tf.reduce_mean(tf.square(reconstruction - inputs))

        self.add_loss(0.05 * recon_loss)
        if training:
            result = self.reconstruction_mean(recon_loss)
            self.add_metric(result)

        return self.out(Z)
```

Hãy cùng xem đoạn mã này:

- Hàm tạo tạo DNN với năm lớp ẩn dày đặc và một lớp đầu ra dày đặc. Chúng ta cũng tạo một số liệu theo luồng Mean để theo dõi lỗi tái tạo trong quá trình huấn luyện.

- Phương thức `build()` tạo một lớp dày đặc bổ sung sẽ được sử dụng để tái tạo đầu vào của mô hình. Nó phải được tạo ở đây vì số đơn vị của nó phải bằng số đầu vào, và số này không xác định trước khi phương thức `build()` được gọi.
- Phương thức `call()` xử lý đầu vào thông qua tất cả năm lớp ẩn, sau đó truyền kết quả qua lớp tái tạo, cái này tạo ra tái tạo.
- Sau đó, phương thức `call()` tính toán mất mát tái tạo (sự khác biệt bình phương trung bình giữa tái tạo và đầu vào), và thêm nó vào danh sách các mất mát của mô hình bằng phương thức `add_loss()`. Lưu ý rằng chúng ta giảm tỷ lệ mất mát tái tạo bằng cách nhân nó với 0.05 (đây là một siêu tham số bạn có thể điều chỉnh). Điều này đảm bảo rằng mất mát tái tạo không chi phối mất mát chính.
- Tiếp theo, chỉ trong quá trình huấn luyện, phương thức `call()` cập nhật số liệu tái tạo và thêm nó vào mô hình để nó có thể được hiển thị. Ví dụ mã này thực sự có thể được đơn giản hóa bằng cách gọi `self.add_metric(recon_loss)` thay thế: Keras sẽ tự động theo dõi trung bình cho bạn.
- Cuối cùng, phương thức `call()` truyền đầu ra của các lớp ẩn đến lớp đầu ra và trả về đầu ra của nó.

Cả tổng mất mát và mất mát tái tạo sẽ giảm trong quá trình huấn luyện:

```
Epoch 1/5
363/363 [=====] - 1s 820us/step - loss: 0.7640 - reconstruction_error: 1.2728
Epoch 2/5
363/363 [=====] - 0s 809us/step - loss: 0.4584 - reconstruction_error: 0.6340
[...]
```

Trong hầu hết các trường hợp, mọi thứ chúng ta đã thảo luận cho đến nay sẽ đủ để triển khai bất kỳ mô hình nào bạn muốn xây dựng, ngay cả với kiến trúc, mất mát và số liệu phức tạp. Tuy nhiên, đối với một số kiến trúc, chẳng hạn như GANs (xem Chương 17), bạn sẽ phải tùy chỉnh vòng lặp huấn luyện. Trước khi đến đó, chúng ta phải xem cách tính toán gradient tự động trong TensorFlow.

Tính toán Gradient bằng Autodiff

Để hiểu cách sử dụng autodiff (xem Chương 10 và Phụ lục B) để tính toán gradient tự động, hãy xem xét một hàm đồ chơi đơn giản:

```
def f(w1, w2):
    return 3 * w1 ** 2 + 2 * w1 * w2
```

Nếu bạn biết giải tích, bạn có thể phân tích thấy rằng đạo hàm riêng của hàm này đối với w_1 là $6w_1 + 2w_2$. Bạn cũng có thể thấy rằng đạo hàm riêng của nó đối với w_2 là $2w_1$. Ví dụ, tại điểm $(w_1, w_2) = (5, 3)$, các đạo hàm riêng này lần lượt bằng 36 và 10, vì vậy vector gradient tại điểm này là $(36, 10)$. Nhưng nếu đây là một mạng nơ-ron, hàm sẽ phức tạp hơn nhiều, thường có hàng chục nghìn tham số, và việc tìm các đạo hàm riêng bằng tay một cách phân tích sẽ là một nhiệm vụ gần như không thể. Một giải pháp có thể là tính toán một xấp xỉ của

mỗi đạo hàm riêng bằng cách đo lường mức độ thay đổi của đầu ra hàm khi bạn điều chỉnh tham số tương ứng một lượng rất nhỏ:

```
>>> w1, w2 = 5, 3

>>> eps = 1e-6

>>> (f(w1 + eps, w2) - f(w1, w2)) / eps
36.000003007075065

>>> (f(w1, w2 + eps) - f(w1, w2)) / eps
10.000000003174137
```

Trông khá đúng! Cách này hoạt động khá tốt và dễ thực hiện, nhưng nó chỉ là một xấp xỉ, và quan trọng là bạn cần gọi `f()` ít nhất một lần cho mỗi tham số (không phải hai lần, vì chúng ta có thể tính `f(w1, w2)` chỉ một lần). Việc phải gọi `f()` ít nhất một lần cho mỗi tham số khiến cách tiếp cận này không khả thi đối với các mạng nơ-ron lớn. Vì vậy, thay vào đó, chúng ta nên sử dụng tự động đạo hàm ngược (reverse-mode autodiff). TensorFlow làm cho điều này khá đơn giản:

```
w1, w2 = tf.Variable(5.), tf.Variable(3.)
with tf.GradientTape() as tape:
    z = f(w1, w2)

gradients = tape.gradient(z, [w1, w2])
```

Đầu tiên chúng ta định nghĩa hai biến `w1` và `w2`, sau đó chúng ta tạo một ngữ cảnh `tf.GradientTape` sẽ tự động ghi lại mọi phép toán liên quan đến một biến, và cuối cùng chúng ta yêu cầu `tape` này tính toán gradient của kết quả `z` đối với cả hai biến `[w1, w2]`. Hãy xem các gradient mà TensorFlow đã tính toán:

```
>>> gradients
[<tf.Tensor: shape=(), dtype=float32, numpy=36.0>,
 <tf.Tensor: shape=(), dtype=float32, numpy=10.0>]
```

Hoàn hảo! Kết quả không chỉ chính xác (độ chính xác chỉ bị giới hạn bởi các lỗi dấu phẩy động), mà phương thức `gradient()` chỉ đi qua các phép tính đã ghi một lần (theo thứ tự ngược lại), bất kể có bao nhiêu biến, vì vậy nó cực kỳ hiệu quả. Thật kỳ diệu!

Tape tự động bị xóa ngay sau khi bạn gọi phương thức `gradient()` của nó, vì vậy bạn sẽ gặp ngoại lệ nếu bạn cố gắng gọi `gradient()` hai lần:

```
with tf.GradientTape() as tape:
    z = f(w1, w2)

dz_dw1 = tape.gradient(z, w1) # trả về tensor 36.0
dz_dw2 = tape.gradient(z, w2) # raises a RuntimeError!
```


Nếu bạn cần gọi `gradient()` nhiều hơn một lần, bạn phải làm cho `tape` bền bỉ và xóa nó mỗi khi bạn dùng xong để giải phóng tài nguyên:

```
with tf.GradientTape(persistent=True) as tape:
    z = f(w1, w2)

dz_dw1 = tape.gradient(z, w1) # trả về tensor 36.0
dz_dw2 = tape.gradient(z, w2) # trả về tensor 10.0, hoạt động tốt!

del tape
```

Theo mặc định, `tape` sẽ chỉ theo dõi các phép toán liên quan đến biến, vì vậy nếu bạn cố gắng tính toán gradient của `z` đối với bất cứ thứ gì không phải là biến, kết quả sẽ là `None`:

```
c1, c2 = tf.constant(5.), tf.constant(3.)
with tf.GradientTape() as tape:
    z = f(c1, c2)

gradients = tape.gradient(z, [c1, c2]) # trả về [None, None]
```

Tuy nhiên, bạn có thể buộc `tape` theo dõi bất kỳ tensor nào bạn thích, để ghi lại mọi phép toán liên quan đến chúng. Sau đó, bạn có thể tính toán gradient đối với các tensor này, như thể chúng là biến:

```
with tf.GradientTape() as tape:
    tape.watch(c1)
    tape.watch(c2)
    z = f(c1, c2)

gradients = tape.gradient(z, [c1, c2]) # trả về [tensor 36., tensor 10.]
```

Điều này có thể hữu ích trong một số trường hợp, chẳng hạn như nếu bạn muốn triển khai một mất mát chính quy hóa phạt các kích hoạt thay đổi nhiều khi đầu vào thay đổi ít: mất mát sẽ dựa trên gradient của các kích hoạt đối với đầu vào. Vì đầu vào không phải là biến, bạn sẽ cần bảo `tape` theo dõi chúng.

Hầu hết thời gian, một `tape gradient` được sử dụng để tính toán gradient của một giá trị duy nhất (thường là mất mát) đối với một tập hợp các giá trị (thường là các tham số mô hình). Đây là nơi tự động đạo hàm ngược tỏa sáng, vì nó chỉ cần thực hiện một lượt tiến và một lượt ngược để nhận tất cả các gradient cùng một lúc. Nếu bạn cố gắng tính toán gradient của một vector, ví dụ một vector chứa nhiều mất mát, thì TensorFlow sẽ tính toán gradient của tổng vector đó. Vì vậy, nếu bạn cần lấy các gradient riêng lẻ (ví dụ: gradient của mỗi mất mát đối với các tham số mô hình), bạn phải gọi phương thức `jacobian()` của `tape`: nó sẽ thực hiện tự động đạo hàm ngược một lần cho mỗi mất mát trong vector (tất cả song song theo mặc định). Thậm chí có thể tính toán đạo hàm riêng cấp hai (Hessian, tức là đạo hàm riêng của đạo hàm riêng), nhưng điều này hiếm khi cần trong thực tế (xem phần “Tính toán Gradient bằng Autodiff” của sổ tay chương này để biết ví dụ).

Trong một số trường hợp, bạn có thể muốn dừng gradient backpropagating qua một phần nào đó của mạng nơ-ron của bạn. Để làm điều này, bạn phải sử dụng hàm `tf.stop_gradient()`. Hàm trả về đầu vào của nó trong lượt tiến (giống như `tf.identity()`), nhưng nó không cho phép gradient đi qua trong backpropagation (nó hoạt động như một hằng số):

```
def f(w1, w2):
    return 3 * w1 ** 2 + tf.stop_gradient(2 * w1 * w2)

with tf.GradientTape() as tape:
    z = f(w1, w2) # lượt tiến không bị ảnh hưởng bởi stop_gradient()

gradients = tape.gradient(z, [w1, w2]) # trả về [tensor 30., None]
```

Cuối cùng, đôi khi bạn có thể gặp phải một số vấn đề số học khi tính toán gradient. Ví dụ, nếu bạn tính toán gradient của hàm căn bậc hai tại $x = 10^{-50}$, kết quả sẽ là vô hạn. Trong thực tế, độ dốc tại điểm đó không phải là vô hạn, nhưng nó lớn hơn khả năng xử lý của các số thập phân 32 bit:

```
>>> x = tf.Variable(1e-50)

>>> with tf.GradientTape() as tape:
...     z = tf.sqrt(x)
...
>>> tape.gradient(z, [x])
[<tf.Tensor: shape=(), dtype=float32, numpy=inf>]
```

Để giải quyết vấn đề này, thường nên thêm một giá trị nhỏ vào x (chẳng hạn như 10^{-6}) khi tính căn bậc hai của nó.

Hàm mũ cũng là một nguồn gây đau đầu thường xuyên, vì nó tăng cực kỳ nhanh. Ví dụ, cách hàm `my_softplus()` được định nghĩa trước đó không ổn định về mặt số học. Nếu bạn tính `my_softplus(100.0)`, bạn sẽ nhận được vô hạn thay vì kết quả chính xác (khoảng 100). Nhưng có thể viết lại hàm để làm cho nó ổn định về mặt số học: hàm `softplus` được định nghĩa là $\log(1 + \exp(z))$, cái này cũng bằng $\log(1 + \exp(-|z|)) + \max(z, 0)$ (xem sổ tay để biết chứng minh toán học) và lợi thế của dạng thứ hai này là số hạng mũ không thể bùng nổ. Vì vậy, đây là một triển khai tốt hơn của hàm `my_softplus()`:

```
def my_softplus(z):
    return tf.math.log(1 + tf.exp(-tf.abs(z))) + tf.maximum(0., z)
```

Trong một số trường hợp hiếm hoi, một hàm ổn định về mặt số học vẫn có thể có các gradient không ổn định về mặt số học. Trong những trường hợp như vậy, bạn sẽ phải cho TensorFlow biết phương trình nào sẽ sử dụng cho các gradient, thay vì để nó sử dụng autodiff. Để làm điều này, bạn phải sử dụng bộ trang trí `@tf.custom_gradient` khi định nghĩa hàm, và trả về cả kết quả thông thường của hàm và một hàm tính toán gradient. Ví dụ, hãy cập nhật hàm `my_softplus()` để cũng trả về một hàm gradient ổn định về mặt số học:

```
@tf.custom_gradient
def my_softplus(z):
    def my_softplus_gradients(grads): # grads = được backprop từ các lớp trên
        return grads * (1 - 1 / (1 + tf.exp(z))) # gradient ổn định của softplus
    result = tf.math.log(1 + tf.exp(-tf.abs(z))) + tf.maximum(0., z)
    return result, my_softplus_gradients
```

Nếu bạn biết giải tích vi phân (xem sổ tay hướng dẫn về chủ đề này), bạn có thể tìm thấy rằng đạo hàm của $\log(1 + \exp(z))$ là $\exp(z)/(1 + \exp(z))$. Nhưng dạng này không ổn định: đối với các giá trị lớn của z , nó kết thúc việc tính toán vô hạn chia cho vô hạn, trả về NaN. Tuy nhiên, với một chút biến đổi đại số, bạn có thể chứng minh rằng nó cũng bằng $1 - 1/(1 + \exp(z))$, cái này ổn định.

Hàm `my_softplus_gradients()` sử dụng phương trình này để tính toán gradient. Lưu ý rằng hàm này sẽ nhận làm đầu vào các gradient đã được backpropagated cho đến hàm `my_softplus()`, và theo quy tắc chuỗi chúng ta phải nhân chúng với gradient của hàm này.

Bây giờ khi chúng ta tính toán gradient của hàm `my_softplus()`, chúng ta nhận được kết quả phù hợp, ngay cả đối với các giá trị đầu vào lớn.

Chúc mừng! Bây giờ bạn có thể tính toán gradient của bất kỳ hàm nào (miễn là nó khả vi tại điểm bạn tính toán), thậm chí chặn backpropagation khi cần, và viết các hàm gradient của riêng bạn! Điều này có lẽ linh hoạt hơn bạn cần, ngay cả khi bạn xây dựng các vòng lặp huấn luyện tùy chỉnh của riêng bạn. Bạn sẽ thấy cách làm điều đó tiếp theo.

Vòng lặp huấn luyện tùy chỉnh

Trong một số trường hợp, phương thức `fit()` có thể không đủ linh hoạt cho những gì bạn cần làm. Ví dụ, bài báo Wide & Deep mà chúng ta đã thảo luận trong Chương 10 sử dụng hai bộ tối ưu hóa khác nhau: một cho đường dẫn rộng và một cho đường dẫn sâu. Vì phương thức `fit()` chỉ sử dụng một bộ tối ưu hóa (bộ tối ưu hóa mà chúng ta chỉ định khi biên dịch mô hình), việc triển khai bài báo này đòi hỏi phải viết vòng lặp tùy chỉnh của riêng bạn.

Bạn cũng có thể muốn viết các vòng lặp huấn luyện tùy chỉnh chỉ đơn giản là để cảm thấy tự tin hơn rằng chúng làm chính xác những gì bạn muốn (có lẽ bạn không chắc chắn về một số chi tiết của phương thức `fit()`). Đôi khi cảm thấy an toàn hơn khi làm mọi thứ rõ ràng. Tuy nhiên, hãy nhớ rằng việc viết một vòng lặp huấn luyện tùy chỉnh sẽ làm cho mã của bạn dài hơn, dễ bị lỗi hơn và khó bảo trì hơn.

Đầu tiên, hãy xây dựng một mô hình đơn giản. Không cần biên dịch nó, vì chúng ta sẽ xử lý vòng lặp huấn luyện thủ công:

```
import tensorflow as tf
import numpy as np

l2_reg = tf.keras.regularizers.l2(0.05)
model = tf.keras.models.Sequential([
```

```
tf.keras.layers.Dense(30, activation="relu", kernel_initializer="he_normal",
                        kernel_regularizer=l2_reg),
tf.keras.layers.Dense(1, kernel_regularizer=l2_reg)
])
```

Tiếp theo, hãy tạo một hàm nhỏ sẽ lấy ngẫu nhiên một batch các thể hiện từ tập huấn luyện (trong Chương 13 chúng ta sẽ thảo luận về API `tf.data`, cái này cung cấp một lựa chọn thay thế tốt hơn nhiều):

```
def random_batch(X, y, batch_size=32):
    idx = np.random.randint(len(X), size=batch_size)
    return X[idx], y[idx]
```

Hãy định nghĩa thêm một hàm sẽ hiển thị trạng thái huấn luyện, bao gồm số bước, tổng số bước, mất mát trung bình kể từ khi bắt đầu epoch (chúng ta sẽ sử dụng số liệu Mean để tính toán nó), và các số liệu khác:

```
def print_status_bar(step, total, loss, metrics=None):
    metrics = " - ".join([f"{m.name}: {m.result():.4f}"
                           for m in [loss] + (metrics or [])])
    end = "" if step < total else "\n"
    print(f"\r{step}/{total} - " + metrics, end=end)
```

Đoạn mã này tự giải thích, trừ khi bạn không quen với định dạng chuỗi Python: `{m.result():.4f}` sẽ định dạng kết quả của số liệu dưới dạng số thập phân với bốn chữ số sau dấu thập phân, và việc sử dụng `\r` (dấu quay về đầu dòng) cùng với `end=""` đảm bảo rằng thanh trạng thái luôn được in trên cùng một dòng.

Với điều đó, hãy bắt tay vào việc! Đầu tiên, chúng ta cần định nghĩa một số siêu tham số và chọn bộ tối ưu hóa, hàm mất mát và các số liệu (chỉ MAE trong ví dụ này):

```
# Giả định X_train_scaled và y_train đã được định nghĩa
# Ví dụ:
# X_train_scaled = np.random.rand(1000, 10).astype(np.float32)
# y_train = np.random.rand(1000, 1).astype(np.float32)

n_epochs = 5
batch_size = 32
n_steps = len(X_train_scaled) // batch_size
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01)
loss_fn = tf.keras.losses.mean_squared_error
mean_loss = tf.keras.metrics.Mean(name="mean_loss")
metrics = [tf.keras.metrics.MeanAbsoluteError()]
```

Và bây giờ chúng ta đã sẵn sàng xây dựng vòng lặp tùy chỉnh!

```
for epoch in range(1, n_epochs + 1):
    print("Epoch {}/{}".format(epoch, n_epochs))
    for step in range(1, n_steps + 1):
```

```

X_batch, y_batch = random_batch(X_train_scaled, y_train)
with tf.GradientTape() as tape:
    y_pred = model(X_batch, training=True)
    main_loss = tf.reduce_mean(loss_fn(y_batch, y_pred))
    loss = tf.add_n([main_loss] + model.losses)

    gradients = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(gradients, model.trainable_variables))
    mean_loss(loss)

    for metric in metrics:
        metric(y_batch, y_pred)

    print_status_bar(step, n_steps, mean_loss, metrics)

for metric in [mean_loss] + metrics:
    metric.reset_states()

```

Có rất nhiều điều đang diễn ra trong đoạn mã này, vì vậy hãy cùng xem xét chi tiết:

- Chúng ta tạo hai vòng lặp lồng nhau: một cho các epoch, một cho các batch trong một epoch.
- Sau đó, chúng ta lấy mẫu một batch ngẫu nhiên từ tập huấn luyện.
- Bên trong khối `tf.GradientTape()`, chúng ta đưa ra dự đoán cho một batch, sử dụng mô hình như một hàm, và chúng ta tính toán mất mát: nó bằng tổng mất mát chính cộng với các mất mát khác (trong mô hình này, có một mất mát chính quy hóa cho mỗi lớp). Vì hàm `mean_squared_error()` trả về một mất mát cho mỗi thể hiện, chúng ta tính trung bình trên batch bằng `tf.reduce_mean()` (nếu bạn muốn áp dụng các trọng số khác nhau cho mỗi thể hiện, đây là nơi bạn sẽ làm điều đó). Các mất mát chính quy hóa đã được giảm xuống thành một đại lượng vô hướng mỗi cái, vì vậy chúng ta chỉ cần cộng chúng lại (sử dụng `tf.add_n()`, cái này cộng nhiều tensor có cùng hình dạng và kiểu dữ liệu).
- Tiếp theo, chúng ta yêu cầu `tape` tính toán gradient của mất mát đối với từng biến có thể huấn luyện — không phải tất cả các biến! — và chúng ta áp dụng chúng cho bộ tối ưu hóa để thực hiện một bước giảm độ dốc.
- Sau đó, chúng ta cập nhật mất mát trung bình và các số liệu (trên epoch hiện tại), và chúng ta hiển thị thanh trạng thái.
- Cuối mỗi epoch, chúng ta đặt lại trạng thái của mất mát trung bình và các số liệu.

Nếu bạn muốn áp dụng cắt gradient (xem Chương 11), hãy đặt siêu tham số `clipnorm` hoặc `clipvalue` của bộ tối ưu hóa. Nếu bạn muốn áp dụng bất kỳ phép biến đổi nào khác cho các gradient, chỉ cần làm như vậy trước khi gọi phương thức `apply_gradients()`. Và nếu bạn muốn thêm ràng buộc trọng số vào mô hình của mình (ví dụ: bằng cách đặt `kernel_constraint` hoặc `bias_constraint` khi tạo một lớp), bạn nên cập nhật vòng lặp huấn luyện để áp dụng các ràng buộc này ngay sau `apply_gradients()`, như sau:

```
for variable in model.variables:
    if variable.constraint is not None:
        variable.assign(variable.constraint(variable))
```

Như bạn có thể thấy, có khá nhiều điều bạn cần làm đúng, và rất dễ mắc lỗi. Nhưng mặt tươi sáng là bạn có toàn quyền kiểm soát.

Bây giờ bạn đã biết cách tùy chỉnh bất kỳ phần nào của mô hình và thuật toán huấn luyện của mình, hãy xem cách bạn có thể sử dụng tính năng tạo biểu đồ tự động của TensorFlow: nó có thể tăng tốc đáng kể mã tùy chỉnh của bạn, và nó cũng sẽ làm cho nó có thể di động sang bất kỳ nền tảng nào được TensorFlow hỗ trợ (xem Chương 19).

Hàm và biểu đồ TensorFlow

Trở lại TensorFlow 1, biểu đồ là không thể tránh khỏi (cũng như sự phức tạp đi kèm với chúng) vì chúng là một phần trung tâm của API của TensorFlow. Kể từ TensorFlow 2 (phát hành năm 2019), biểu đồ vẫn ở đó, nhưng không còn là trung tâm, và chúng đơn giản hơn nhiều (rất nhiều!) để sử dụng. Để cho thấy sự đơn giản như thế nào, hãy bắt đầu với một hàm tầm thường tính toán lập phương của đầu vào của nó:

```
def cube(x):
    return x**3
```

Chúng ta rõ ràng có thể gọi hàm này với một giá trị Python, chẳng hạn như một số nguyên hoặc một số thập phân, hoặc chúng ta có thể gọi nó với một tensor:

```
>>> cube(2)
8

>>> cube(tf.constant(2.0))
<tf.Tensor: shape=(), dtype=float32, numpy=8.0>
```

Bây giờ, hãy sử dụng `tf.function()` để chuyển đổi hàm Python này thành một hàm TensorFlow:

```
>>> tf_cube = tf.function(cube)

>>> tf_cube
<tensorflow.python.eager.def_function.Function at 0x7fbfe0c54d50>
```

Hàm TF này sau đó có thể được sử dụng chính xác như hàm Python gốc, và nó sẽ trả về cùng một kết quả (nhưng luôn dưới dạng tensor):

```
>>> tf_cube(2)
<tf.Tensor: shape=(), dtype=int32, numpy=8>

>>> tf_cube(tf.constant(2.0))
<tf.Tensor: shape=(), dtype=float32, numpy=8.0>
```

Dưới nắp, `tf.function()` đã phân tích các phép tính được thực hiện bởi hàm `cube()` và tạo ra một biểu đồ tính toán tương đương! Như bạn có thể thấy, nó khá dễ dàng (chúng ta sẽ xem xét cách hoạt động này ngay sau đây).

Ngoài ra, chúng ta có thể đã sử dụng `tf.function` làm một decorator; điều này thực ra phổ biến hơn:

```
@tf.function
def tf_cube(x):
    return x**3
```

Hàm Python gốc vẫn có sẵn thông qua thuộc tính `python_function` của hàm TF, trong trường hợp bạn cần nó:

```
>>> tf_cube.python_function(2)
8
```

TensorFlow tối ưu hóa biểu đồ tính toán, cắt bỏ các nút không sử dụng, đơn giản hóa các biểu thức (ví dụ: `1 + 2` sẽ được thay thế bằng `3`), v.v. Khi biểu đồ được tối ưu hóa đã sẵn sàng, hàm TF thực thi hiệu quả các phép toán trong biểu đồ, theo đúng thứ tự (và song song khi có thể). Kết quả là, một hàm TF thường sẽ chạy nhanh hơn nhiều so với hàm Python gốc, đặc biệt nếu nó thực hiện các phép tính phức tạp. Hầu hết thời gian bạn sẽ không thực sự cần biết nhiều hơn thế: khi bạn muốn tăng cường một hàm Python, chỉ cần biến nó thành một hàm TF. Thế thôi!

Hơn nữa, nếu bạn đặt `jit_compile=True` khi gọi `tf.function()`, thì TensorFlow sẽ sử dụng đại số tuyến tính tăng tốc (XLA) để biên dịch các kernel chuyên dụng cho biểu đồ của bạn, thường hợp nhất nhiều phép toán. Ví dụ, nếu hàm TF của bạn gọi `tf.reduce_sum(a * b + c)`, thì không có XLA, hàm trước tiên sẽ cần tính `a * b` và lưu kết quả vào một biến tạm thời, sau đó thêm `c` vào biến đó, và cuối cùng gọi `tf.reduce_sum()` trên kết quả. Với XLA, toàn bộ phép tính được biên dịch thành một kernel duy nhất, cái này sẽ tính `tf.reduce_sum(a * b + c)` trong một lần, mà không sử dụng bất kỳ biến tạm thời lớn nào. Điều này không chỉ nhanh hơn nhiều, mà còn sử dụng ít RAM hơn đáng kể.

Khi bạn viết một hàm mất mát tùy chỉnh, một số liệu tùy chỉnh, một lớp tùy chỉnh hoặc bất kỳ hàm tùy chỉnh nào khác và bạn sử dụng nó trong một mô hình Keras (như chúng ta đã làm trong suốt chương này), Keras tự động chuyển đổi hàm của bạn thành một hàm TF — không cần sử dụng `tf.function()`. Vì vậy, hầu hết thời gian, điều kỳ diệu là hoàn toàn minh bạch. Và nếu bạn muốn Keras sử dụng XLA, bạn chỉ cần đặt `jit_compile=True` khi gọi phương thức `compile()`. Dễ dàng!

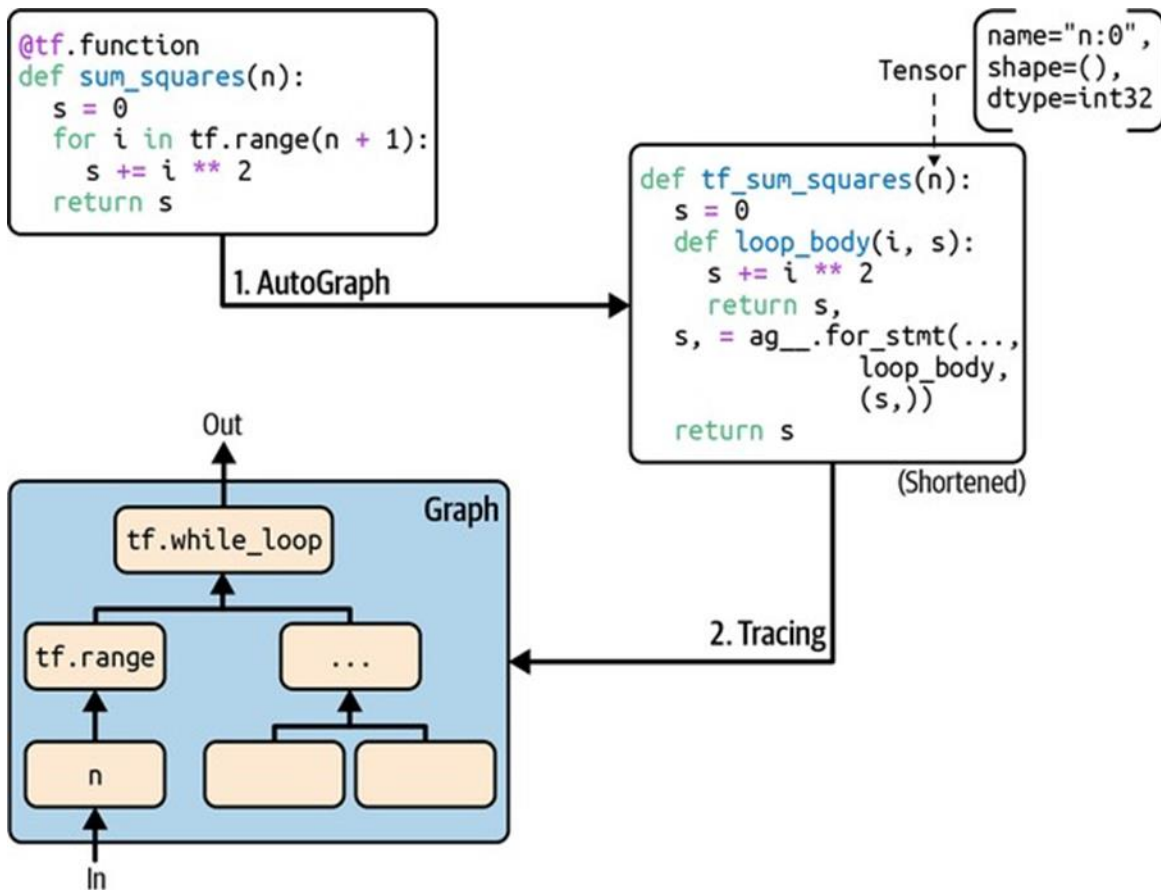
Theo mặc định, một hàm TF tạo một biểu đồ mới cho mỗi tập hợp hình dạng và kiểu dữ liệu đầu vào duy nhất và lưu trữ nó để gọi lại sau này. Ví dụ, nếu bạn gọi `tf_cube(tf.constant(10))`, một biểu đồ sẽ được tạo cho các tensor `int32` có hình dạng `[]`. Sau đó, nếu bạn gọi `tf_cube(tf.constant(20))`, cùng một biểu đồ sẽ được tái sử dụng. Nhưng nếu bạn sau đó gọi `tf_cube(tf.constant([10, 20]))`, một biểu đồ mới sẽ được tạo

cho các tensor `int32` có hình dạng `[2]`. Đây là cách các hàm TF xử lý tính đa hình (tức là các kiểu và hình dạng đối số thay đổi).

Tuy nhiên, điều này chỉ đúng đối với các đối số tensor: nếu bạn truyền các giá trị số Python cho một hàm TF, một biểu đồ mới sẽ được tạo cho mỗi giá trị riêng biệt: ví dụ, gọi `tf_cube(10)` và `tf_cube(20)` sẽ tạo ra hai biểu đồ.

AutoGraph và Tracing

Vậy TensorFlow tạo biểu đồ như thế nào? Nó bắt đầu bằng cách phân tích mã nguồn của hàm Python để nắm bắt tất cả các câu lệnh điều khiển luồng, chẳng hạn như vòng lặp `for`, vòng lặp `while`, và câu lệnh `if`, cũng như các câu lệnh `break`, `continue`, và `return`. Bước đầu tiên này được gọi là **AutoGraph**. Lý do TensorFlow phải phân tích mã nguồn là vì Python không cung cấp bất kỳ cách nào khác để nắm bắt các câu lệnh điều khiển luồng: nó cung cấp các phương thức “magic” như `__add__()` và `__mul__()` để nắm bắt các toán tử như `+` và `*`, nhưng không có phương thức magic `__while__()` hoặc `__if__()`. Sau khi phân tích mã của hàm, AutoGraph xuất ra một phiên bản nâng cấp của hàm đó, trong đó tất cả các câu lệnh điều khiển luồng được thay thế bằng các phép toán TensorFlow thích hợp, chẳng hạn như `tf.while_loop()` cho vòng lặp `for` và `tf.cond()` cho câu lệnh `if`. Ví dụ, trong Hình 12-4, AutoGraph phân tích mã nguồn của hàm Python `sum_squares()`, và nó tạo ra hàm `tf_sum_squares()`. Trong hàm này, vòng lặp `for` được thay thế bằng định nghĩa của hàm `loop_body()` (chứa thân của vòng lặp `for` gốc), sau đó là một lệnh gọi đến hàm `for_stmt()`. Lệnh gọi này sẽ xây dựng phép toán `tf.while_loop()` thích hợp trong biểu đồ tính toán.



Hình 12-4. Cách TensorFlow tạo biểu đồ bằng AutoGraph và tracing

Tiếp theo, TensorFlow gọi hàm “nâng cấp” này, nhưng thay vì truyền đối số, nó truyền một **tensor biểu tượng** — một tensor không có bất kỳ giá trị thực nào, chỉ có tên, kiểu dữ liệu và hình dạng. Ví dụ, nếu bạn gọi `sum_squares(tf.constant(10))`, thì hàm `tf_sum_squares()` sẽ được gọi với một tensor biểu tượng kiểu `int32` và hình dạng `[]`. Hàm sẽ chạy ở **chế độ biểu đồ (graph mode)**, nghĩa là mỗi phép toán TensorFlow sẽ thêm một nút vào biểu đồ để đại diện cho chính nó và các tensor đầu ra của nó (trái ngược với chế độ thông thường, được gọi là **thực thi tức thì (eager execution)**, hoặc **chế độ tức thì (eager mode)**). Trong chế độ biểu đồ, các phép toán TF không thực hiện bất kỳ phép tính nào. Chế độ biểu đồ là chế độ mặc định trong TensorFlow 1. Trong Hình 12-4, bạn có thể thấy hàm `tf_sum_squares()` được gọi với một tensor biểu tượng làm đối số (trong trường hợp này, một tensor `int32` có hình dạng `[]`) và biểu đồ cuối cùng được tạo ra trong quá trình tracing. Các nút đại diện cho các phép toán, và các mũi tên đại diện cho các tensor (cả hàm được tạo và biểu đồ đều được đơn giản hóa).

Quy tắc hàm TF

Hầu hết thời gian, việc chuyển đổi một hàm Python thực hiện các phép toán TensorFlow thành một hàm TF rất đơn giản: trang trí nó bằng `@tf.function` hoặc để Keras xử lý cho bạn. Tuy nhiên, có một vài quy tắc cần tuân thủ:

- Nếu bạn gọi bất kỳ thư viện bên ngoài nào, bao gồm NumPy hoặc thậm chí thư viện chuẩn, lệnh gọi này sẽ chỉ chạy trong quá trình tracing; nó sẽ không phải là một phần của biểu đồ. Thật vậy, một biểu đồ TensorFlow chỉ có thể bao gồm các cấu trúc TensorFlow (tensor, phép toán, biến, tập dữ liệu, v.v.). Vì vậy, hãy đảm bảo bạn sử dụng `tf.reduce_sum()` thay vì `np.sum()`, `tf.sort()` thay vì hàm `sorted()` tích hợp sẵn, v.v. (trừ khi bạn thực sự muốn mã chỉ chạy trong quá trình tracing). Điều này có một vài ý nghĩa bổ sung:
 - Nếu bạn định nghĩa một hàm TF $f(x)$ chỉ trả về `np.random.rand()`, một số ngẫu nhiên sẽ chỉ được tạo khi hàm được traced, vì vậy `f(tf.constant(2.))` và `f(tf.constant(3.))` sẽ trả về cùng một số ngẫu nhiên, nhưng `f(tf.constant([2., 3.]))` sẽ trả về một số khác. Nếu bạn thay thế `np.random.rand()` bằng `tf.random.uniform([])`, thì một số ngẫu nhiên mới sẽ được tạo ra sau mỗi lần gọi, vì phép toán sẽ là một phần của biểu đồ.
 - Nếu mã không phải TensorFlow của bạn có các tác dụng phụ (chẳng hạn như ghi nhật ký một cái gì đó hoặc cập nhật bộ đếm Python), thì bạn không nên mong đợi những tác dụng phụ đó xảy ra mỗi khi bạn gọi hàm TF, vì chúng sẽ chỉ xảy ra khi hàm được traced.
 - Bạn có thể bọc mã Python tùy ý trong một phép toán `tf.py_function()`, nhưng làm như vậy sẽ cản trở hiệu suất, vì TensorFlow sẽ không thể thực hiện bất kỳ tối ưu hóa biểu đồ nào trên mã này. Nó cũng sẽ làm giảm tính di động, vì biểu đồ sẽ chỉ chạy trên các nền tảng nơi Python có sẵn (và nơi các thư viện phù hợp được cài đặt).
- Bạn có thể gọi các hàm Python hoặc hàm TF khác, nhưng chúng phải tuân theo các quy tắc tương tự, vì TensorFlow sẽ nắm bắt các phép toán của chúng trong biểu đồ tính toán. Lưu ý rằng các hàm khác này không cần phải được trang trí bằng `@tf.function`.
- Nếu hàm tạo một biến TensorFlow (hoặc bất kỳ đối tượng TensorFlow có trạng thái nào khác, chẳng hạn như tập dữ liệu hoặc hàng đợi), nó phải làm như vậy ngay từ lần gọi đầu tiên, và chỉ sau đó, nếu không bạn sẽ gặp ngoại lệ. Thường thì nên tạo biến bên ngoài hàm TF (ví dụ: trong phương thức `build()` của một lớp tùy chỉnh). Nếu bạn muốn gán một giá trị mới cho biến, hãy đảm bảo bạn gọi phương thức `assign()` của nó thay vì sử dụng toán tử `=`.
- Mã nguồn của hàm Python của bạn phải có sẵn cho TensorFlow. Nếu mã nguồn không có sẵn (ví dụ: nếu bạn định nghĩa hàm của mình trong shell Python, cái này không cấp quyền truy cập vào mã nguồn, hoặc nếu bạn chỉ triển khai các tệp Python `.pyc` đã biên dịch vào sản xuất), thì quá trình tạo biểu đồ sẽ thất bại hoặc có chức năng hạn chế.
- TensorFlow sẽ chỉ nắm bắt các vòng lặp `for` lặp lại trên một tensor hoặc một `tf.data.Dataset` (xem Chương 13). Do đó, hãy đảm bảo bạn sử dụng `for i in tf.range(x)` thay vì `for i in range(x)`, nếu không vòng lặp sẽ không được nắm bắt trong biểu đồ. Thay vào đó, nó sẽ chạy trong quá trình tracing. (Đây có thể là điều bạn muốn nếu vòng lặp `for` có ý nghĩa xây dựng biểu đồ; ví dụ: để tạo từng lớp trong mạng nơ-ron.)

- Như mọi khi, vì lý do hiệu suất, bạn nên ưu tiên triển khai vector hóa bất cứ khi nào có thể, thay vì sử dụng vòng lặp.

Đã đến lúc tóm tắt! Trong chương này, chúng ta bắt đầu với tổng quan ngắn gọn về TensorFlow, sau đó chúng ta xem xét API cấp thấp của TensorFlow, bao gồm tensor, phép toán, biến và các cấu trúc dữ liệu đặc biệt. Sau đó, chúng ta sử dụng các công cụ này để tùy chỉnh hầu hết mọi thành phần trong API Keras. Cuối cùng, chúng ta xem xét cách các hàm TF có thể tăng hiệu suất, cách các biểu đồ được tạo bằng AutoGraph và tracing, và các quy tắc cần tuân theo khi bạn viết các hàm TF (nếu bạn muốn mở hộp đen thêm một chút và khám phá các biểu đồ được tạo, bạn sẽ tìm thấy các chi tiết kỹ thuật trong Phụ lục D).

Trong chương tiếp theo, chúng ta sẽ xem xét cách tải và tiền xử lý dữ liệu hiệu quả với TensorFlow.

Bài tập

1. Bạn sẽ mô tả TensorFlow trong một câu ngắn gọn như thế nào? Các tính năng chính của nó là gì? Bạn có thể kể tên các thư viện học sâu phổ biến khác không?
2. TensorFlow có phải là một sự thay thế trực tiếp cho NumPy không? Sự khác biệt chính giữa hai cái là gì?
3. Bạn có nhận được kết quả tương tự với `tf.range(10)` và `tf.constant(np.arange(10))` không?
4. Bạn có thể kể tên sáu cấu trúc dữ liệu khác có sẵn trong TensorFlow, ngoài các tensor thông thường không?
5. Bạn có thể định nghĩa một hàm mất mát tùy chỉnh bằng cách viết một hàm hoặc bằng cách kế thừa từ lớp `tf.keras.losses.Loss`. Bạn sẽ sử dụng mỗi tùy chọn khi nào?
6. Tương tự, bạn có thể định nghĩa một số liệu tùy chỉnh trong một hàm hoặc là một lớp con của `tf.keras.metrics.Metric`. Bạn sẽ sử dụng mỗi tùy chọn khi nào?
7. Khi nào bạn nên tạo một lớp tùy chỉnh so với một mô hình tùy chỉnh?
8. Một số trường hợp sử dụng nào yêu cầu viết vòng lặp huấn luyện tùy chỉnh của riêng bạn?
9. Các thành phần Keras tùy chỉnh có thể chứa mã Python tùy ý, hay chúng phải có thể chuyển đổi sang các hàm TF?
10. Các quy tắc chính cần tuân thủ nếu bạn muốn một hàm có thể chuyển đổi thành hàm TF là gì?
11. Khi nào bạn cần tạo một mô hình Keras động? Bạn làm điều đó như thế nào? Tại sao không làm cho tất cả các mô hình của bạn động?
12. Triển khai một lớp tùy chỉnh thực hiện chuẩn hóa lớp (chúng ta sẽ sử dụng loại lớp này trong Chương 15): a. Phương thức `build()` nên định nghĩa hai trọng số có thể huấn luyện α và β , cả hai đều có hình dạng `input_shape[-1:]` và kiểu dữ liệu `tf.float32`. α nên được khởi tạo bằng 1, và β bằng 0. b. Phương thức `call()` nên tính toán trung bình μ và độ lệch chuẩn σ của các đặc trưng của mỗi thể hiện. Đối với điều này, bạn có thể sử dụng `tf.nn.moments(inputs, axes=-1, keepdims=True)`, cái này trả về trung bình μ và phương sai σ^2 của tất cả các thể hiện (tính căn bậc hai của

phương sai để lấy độ lệch chuẩn). Sau đó, hàm nên tính toán và trả về $\alpha \otimes (X - \mu) / (\sigma + \epsilon) + \beta$, trong đó $\otimes$ đại diện cho phép nhân từng phần tử (*) và ϵ là một số hạng làm mượt (một hằng số nhỏ để tránh chia cho 0, ví dụ: 0.001). c. Đảm bảo rằng lớp tùy chỉnh của bạn tạo ra cùng một đầu ra (hoặc rất gần) như lớp `tf.keras.layers.LayerNormalization`.

13. Huấn luyện một mô hình bằng cách sử dụng vòng lặp huấn luyện tùy chỉnh để giải quyết tập dữ liệu Fashion MNIST (xem Chương 10): a. Hiển thị epoch, lần lặp, mất mát huấn luyện trung bình, và độ chính xác trung bình trên mỗi epoch (được cập nhật ở mỗi lần lặp), cũng như mất mát xác thực và độ chính xác ở cuối mỗi epoch. b. Thử sử dụng một bộ tối ưu hóa khác với tốc độ học khác nhau cho các lớp trên và các lớp dưới.

Các giải pháp cho các bài tập này có sẵn ở cuối sổ tay của chương này, tại <https://homl.info/colab3>.